“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재9

강사 이영배

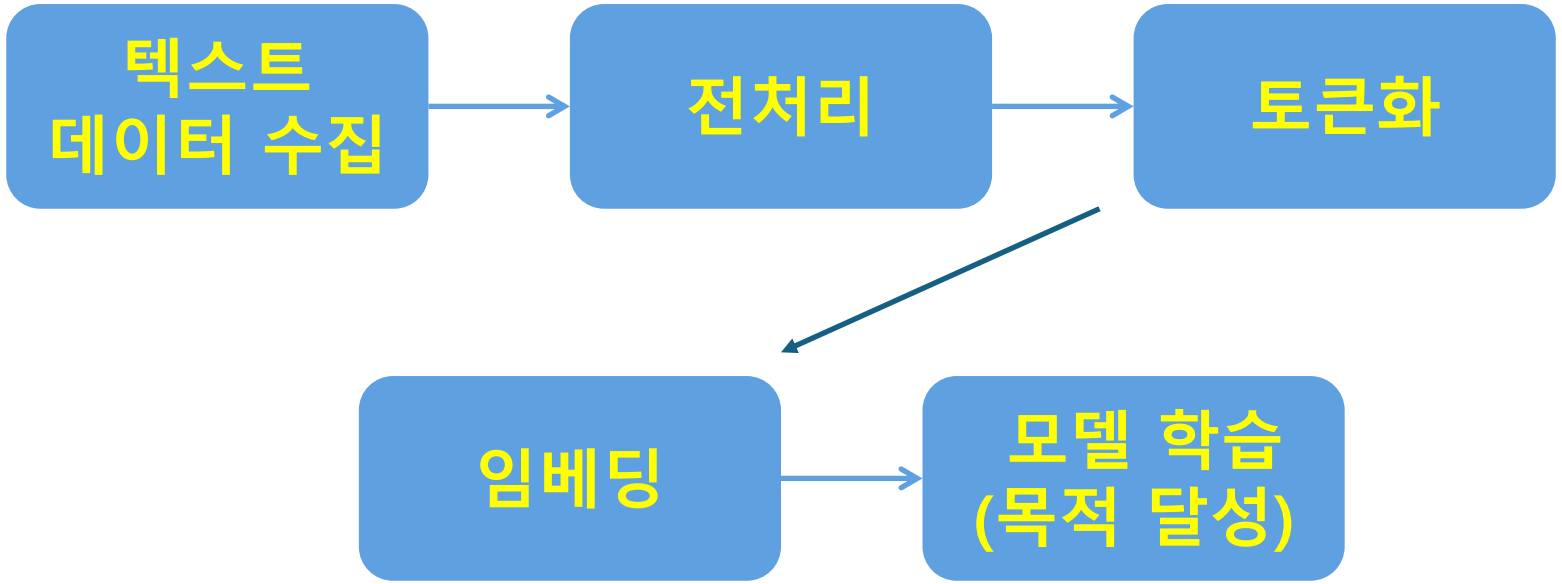
20. 자연어 데이터 준비

자연어

1. 개념 : **일상생활에서 사용하는 언어 예) 한글, 영어 등**
2. 반대말 : **인공 언어 또는 프로그래밍 언어**
3. 자연어 처리(Natural Language Processing, NLP)
   : 인공지능의 한 분야로, 컴퓨터가 인간의 언어를 이해하고 생성하도록 하는 기술
4. 언어 모델(Language Model, LM)
   : NLP 기술을 구현, 문맥을 파악해 다음 단어를 맞추는 알고리즘



자연어 처리 과정



텍스트 데이터 수집



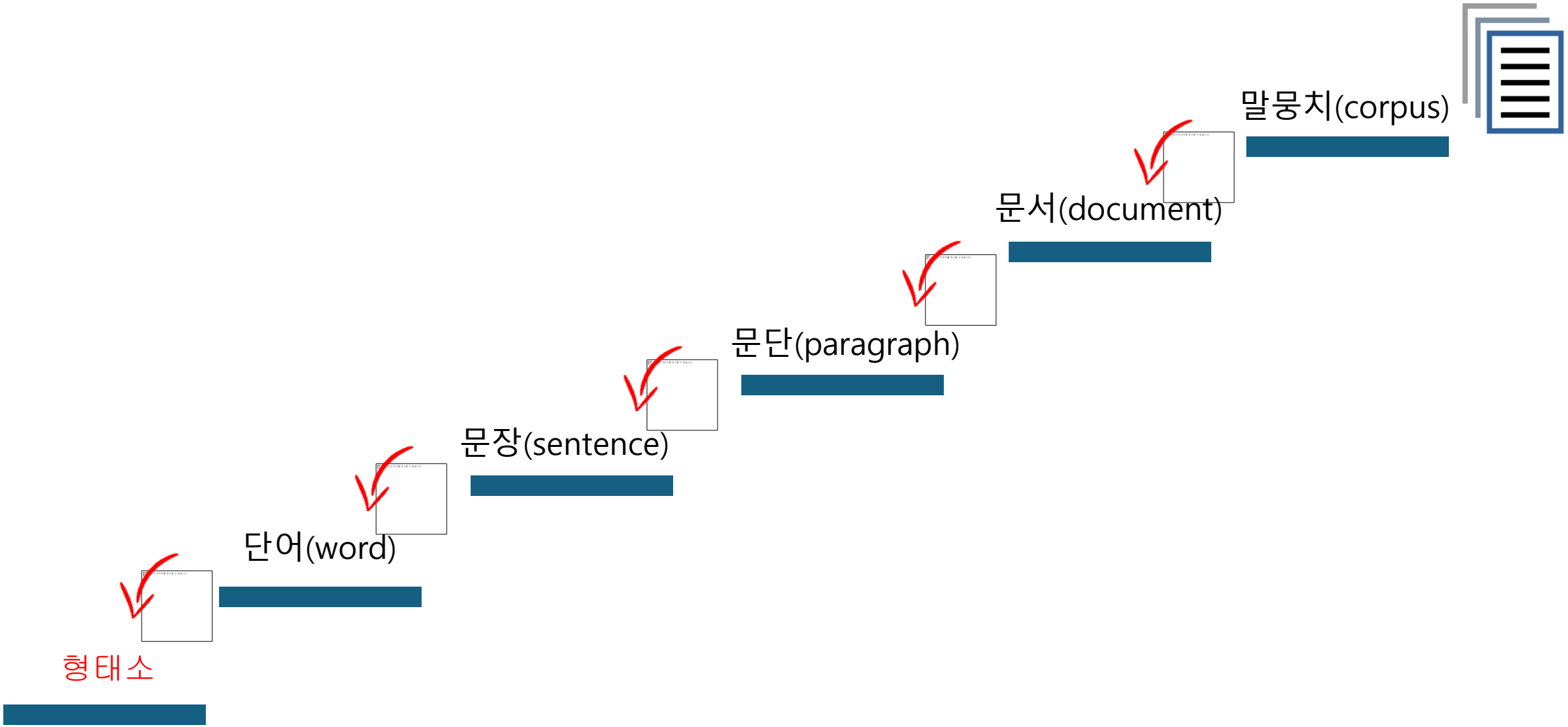
- 공공데이터
- 오픈 API를 이용한 데이터 수집
- 크롤링을 이용한 Web 데이터 수집(SNS/블로그/카페 등)

텍스트 전처리



- 텍스트 데이터에서 불필요한 데이터를 처리하는 작업
- 문장 부호 등 특수문자 제거
- 불용어(stopword) 제거 : 데이터 분석에서 큰 의미가 없는 단어 제거 (예) be 동사 등

텍스트 데이터의 구조와 구성 성분



형태소

1. 개념 : 뜻을 가지고 있는 말 중 가장 작은 단위, 더 나눌 수 없는 단위

2. 예시

1) 영어 : kindness = kind(형용사) + -ness(접사), useful = use(동사) + -ful(접사)

2) 한글 : 책가방 = 책(명사) + 가방(명사), 풋사과 = 풋(접사)- + 사과(명사)

※ 참고
1) 형태소와 단어 : https://terms.naver.com/entry.naver?docId=3559670&cid=58583&categoryId=58729
2) 형태소와 품사 : https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=zzangdol57&logNo=221533747352

영어와 한글의 불용어 차이

1. 영어 : 불용어 → 단어(대명사, 관사, 접속사, 조동사 등)
2. 한글 : 불용어 → 조사, 어미 등 형태소
   ① 꽃밭에 꽃이 많이 피었다
   : 꽃밭(명사)+에(조사), 꽃(명사)+이(조사), 많이(부사), 피었다(동사)
   ② 용언의 경우 1단어가 어간 + 어미의 형태로 2개 이상의 형태소로 구성되는 경우가 많음
   : 피었다 → 피 (어간)/었 (어미)/다 (어미)

토큰화

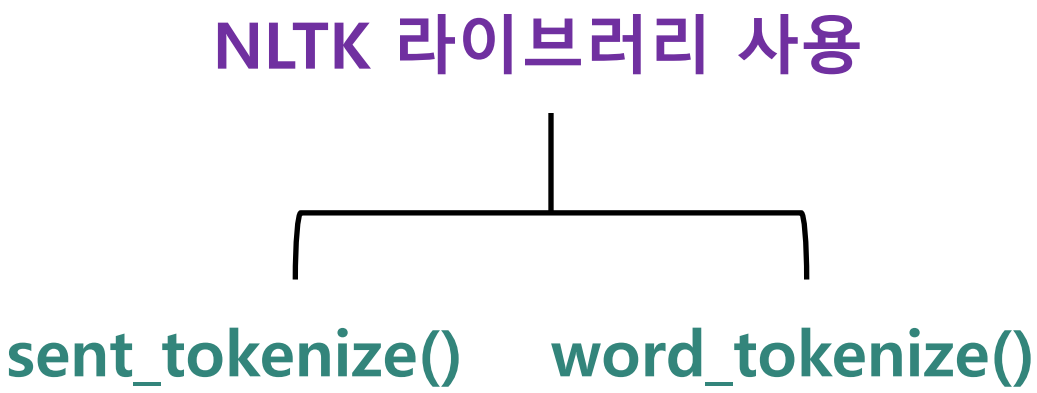


- 토큰화 (tokenization) : 텍스트를 의미 있는 단위로 나누는 작업
- 문장 토큰화 : 문장의 끝을 나타내는 구두점 기호(.,!,?)를 기준으로 분할
- 단어 토큰화 : 띄어쓰기 기준으로 분할
- 형태소 토큰화 : 형태소 단위로 분할

영어와 한글의 토큰화 차이

영어 토큰화	한글 토큰화
문장 / 띄어쓰기(단어) 기준	문장 / 형태소(morpheme) 기준

NLTK 라이브러리를 이용한 텍스트 토큰화(tokenization)



NLTK 라이브러리 실습

1. 라이브러리 임폴트

   import nltk

2. 필요한 기능 추가 다운로드

   ① 문장을 구분하기 위한 문장 부호 다운로드

   nltk.download('punkt_tab')

   ② 불용어 다운로드

   nltk.download('stopwords')

   ③ 불용어 생성 함수 호출 → 영어 불용어 리스트 생성

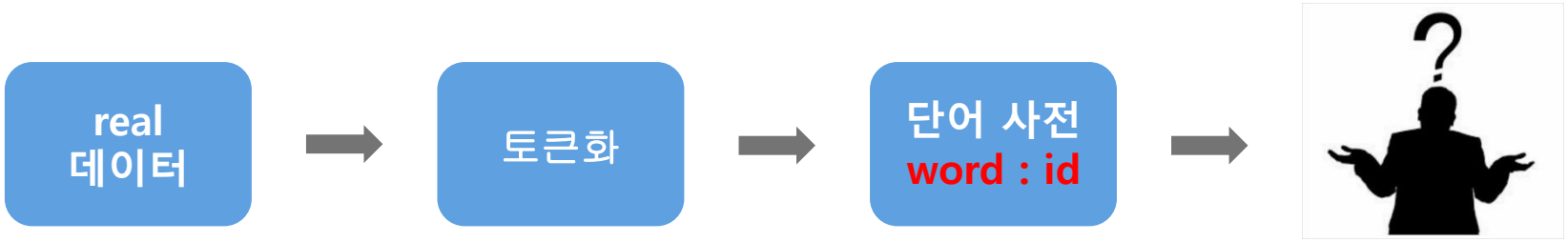
   stopwords_list = nltk.corpus.stopwords.words('english')

3. 문장 단위 토큰화

4. 단어 단위 토큰화

단어 단위 토큰화(tokenization)의 문제점

- OOV(Out-Of-Vocabulary) 문제 발생



(예시) '가사' 감정 분류

"잘못 씹다가 이빨 다나 갈 수 있어",
"넌 친구도 아니야", "디스" 등
가사에서 분노를 표출하는 단어가 반복되는 경우

등장한 단어들 중 '씹다', '이빨', '디스' 등 분노를 표출하는 단어들이 OOV의 가능성이 큼

"분노"감정이 아닌 다른 감정으로 분류될 수 있음

단어 단위 토큰화(tokenization)의 문제점

- 문제점을 완화시킬 수 있는 새로운 토큰화 방법 필요

  1) solution : 서브워드 단위 토큰화(subword tokenization)

  2) 서브워드

  ① 단어(Word)를 더 쪼갠 의미를 가진 최소한의 글자 조각

  ② 필요성

  - 컴퓨터가 인간의 언어를 처리할 때, 신조어를 포함해서 세상의 모든 단어를 다 외우는 것은 불가능하다.
  - 서브워드는 컴퓨터가 적은 양의 단어 조각만 가지고도 조합하여 수억 개의 다양한 문장을 이해하고 처리할 수 있도록 돕는 레고 블록과 같다.



단어 단위 토큰화(tokenization)의 문제점

③ 영어 예시: unhappiness

- 토큰화 결과: un, happi, ness
- 해석: 컴퓨터의 단어 사전에 'unhappiness'라는 긴 단어가 없어도, 부정의 의미를 가진 un, 행복의 어근인 happi, 명사를 만드는 ness를 조합해 "행복하지 않은 상태"라고 스스로 유추해 낼 수 있다.

3) 대표적 알고리즘 : BPE(Byte Pair Encoding), WordPiece (워드피스)

BPE(Byte Pair Encoding)

- BPE(Byte Pair Encoding) 예시
  - 텍스트 데이터를 토큰화

토큰	빈도
hug	10
pug	5
pun	12
bun	4
hugs	5

  - 모든 단어들을 문자 단위로 분리

1) hug → h, u, g / pug → p, u, g / pun → p, u, n / bun → b, u, n / hugs → h, u, g, s

2) b, g, h, n, p, s, u 7개 문자로 모든 토큰 표현 가능 → 7개 문자 → 초기 단어 사전 생성

BPE(Byte Pair Encoding)

- 각 토큰을 단어 사전의 문자로 표현

토큰	빈도
h, u, g	10
p, u, g	5
p, u, n	12
b, u, n	4
h, u, g, s	5

BPE(Byte Pair Encoding)

- 각 문자를 두 개(bigram) 씩 묶어 나열

바이그램 쌍	빈도
h, u	10
u, g	10
p, u	5
u, g	5
p, u	12
u, n	12
b, u	4
u, n	4
h, u	5
u, g	5
g, s	5

BPE(Byte Pair Encoding)

- 바이그램 쌍이 같은 것끼리 그 빈도를 합친다.

바이그램 쌍	빈도
b, u	4
g, s	5
h, u	15
p, u	17
u, g	**20**
u, n	16

- 가장 많이 등장한 바이그램 쌍은 u, g로 총 20회 → **u와 g를 합친 ug를 단어 사전에 추가**
- **단어 사전 업데이트** : b, g, h, n, p, s, u, **ug**

BPE(Byte Pair Encoding)

- 각 토큰을 새로운 단어 사전에 맞게 다시 표시

토큰	빈도
h, **ug**	10
p, **ug**	5
p, u, n	12
b, u, n	4
h, **ug**, s	5

BPE(Byte Pair Encoding)

- 바이그램 쌍 빈도 계산

토큰	빈도
h, **ug**	10
p, **ug**	5
p, u	12
u, n	12
b, u	4
u, n	4
h, **ug**	5
ug, s	5



토큰	빈도
h, **ug**	15
p, **ug**	5
p, u	12
u, n	**16**
b, u	4
ug, s	5

BPE(Byte Pair Encoding)

- 가장 많이 등장한 바이그램 쌍은 u, n으로 총 16회 → **u와 n을 합친 un을 단어 사전에 추가**
- **단어 사전 업데이트** : b, g, h, n, p, s, u, ug, **un**
- 각 토큰을 새로운 단어 사전에 맞게 다시 표시

토큰	빈도
h, ug	10
p, ug	5
p, **un**	12
b, **un**	4
h, ug, s	5

BPE(Byte Pair Encoding)

- 바이그램 쌍 빈도 계산

토큰	빈도
h, ug p, ug p, **un** b, **un** h, ug ug, s	10 5 12 4 5 5



토큰	빈도
h, ug p, ug p, un b, un ug, s	**15** 5 12 4 5

BPE(Byte Pair Encoding)

- 가장 많이 등장한 바이그램 쌍은 h, ug로 총 15회 → **h와 ug을 합친 hug를 단어 사전에 추가**
- **단어 사전 업데이트** : b, g, h, n, p, s, u, ug, un, **hug**
- 각 토큰을 새로운 단어 사전에 맞게 다시 표시

토큰	빈도
p, ug p, un b, un **hug**, s	5 12 4 5

- 바이그램 쌍 빈도 계산

토큰	빈도
p, ug **p, un** b, un hug, s	5 **12** 4 5

BPE(Byte Pair Encoding)

- 가장 많이 등장한 바이그램 쌍은 p, un으로 총 12회

  → **p와 un을 합친 pun를 단어 사전에 추가**

- **단어 사전 업데이트** : b, g, h, n, p, s, u, ug, un, hug, **pun**
- 각 토큰을 새로운 단어 사전에 맞게 다시 표시

토큰	빈도
p, ug b, un hug, s	5 4 5

- 원하는 어휘 집합의 크기가 될 때까지 반복
- 자주 등장하지 않는 단어 pug, bun, hugs는 서브 워드로 분리 → 단어 사전에 포함
- **최종 단어 사전** : b, g, h, n, p, s, u, ug, un, hug, pun
- BPE 알고리즘 → 빈도수가 가장 큰 바이그램 쌍을 병합하는 방식으로 단어 사전 구축
- 토큰화 대상 문장 입력 → 단어 사전 참조 → 단어, 서브워드, 문자로 토큰화

WordPiece

- 학습 과정

  1) 가장 빈번하게 등장하는 바이그램 쌍을 반복적으로 병합, 단어 사전 구성

  2) 등장 확률이 낮은 단어는 단어 사전에 포함되지 않고, 서브워드로 분할되어 단어 사전에 포함

- 서브워드의 표시

  1) 두 개의 해시 태그(#)를 사용

  2) 예시) 너와 함께한 시간 모두 눈부셨다

  3) 토큰화 결과: 너', '##와', ' 함께', '##한', ' 시간', ' 모두', ' 눈부', '##셔', '##ㅆ다'

  4) 전체 토큰의 3가지 유형

     ① 독립적인 단어 토큰: 시간, 모두

     ② 서브워드로서 단어의 시작점 (접두어 형태) 토큰: 너, 함께, 눈부

     ③ 서브워드 (접미어 형태) 토큰: ##와, ##한, ##셔, ##ㅆ다

     → 앞의 토큰에 종속되어 단어를 완성하는 정통적인 서브워드

     ④ 서브워드이지만 단어의 가장 앞부분에 사용된 단어의 시작점 형태의 토큰에는 해시 태그가 붙지 않음

WordPiece

5) 단어의 가장 앞부분에 사용된 단어의 시작점 형태의 서브워드에는 해시 태그가 붙지 않는 이유

① 새로운 단어(어절, 띄어쓰기 단위)의 시작점임을 표시하기 위함

② WordPiece 알고리즘은 문장을 원래대로 복원할 때, ## 기호가 없는 토큰을 기준으로 삼아 띄어쓰기(공백)를 넣는다.

③ 복원 규칙

- ##이 없는 토큰을 만나면 → 앞 토큰과 한 칸 띄우고 쓴다.
- ##이 있는 토큰을 만나면 → ##을 지우고 앞 토큰에 바짝 붙여서 쓴다.
- 복원 규칙 적용 예시
  - 토큰 배열: 너, ##와, 함께, ##한
  - 너 → 첫 토큰이므로 그냥 씀 → 결과: 너
  - ##와 → ##이 있네? 앞 토큰에 붙임 → 결과: 너와
  - 함께 → ##이 없네? 한 칸 띄우고 씀 → 결과: 너와 함께
  - ##한 → ##이 있네? 앞 토큰에 붙임 → 결과: 너와 함께한

WordPiece

- 서브워드 토큰화의 장점

1) OOV (Out-of-Vocabulary) 해결: 사전에 없는 단어나 오타, 신조어가 나와도 글자 조각으로 분해하여 어떻게든 읽어내고 처리한다.

2) 가벼운 모델 용량: 수백만 개의 단어를 다 저장하는 대신, 수만 개의 핵심 서브워드 조각만 저장하므로 컴퓨터 메모리를 극도로 아낄 수 있다.

③ 의미적 연관성 학습: walk, walking, walked를 완전히 다른 단어로 인식하던 과거와 달리, walk라는 공통 서브워드를 공유함으로써 단어 간의 뿌리가 같음을 컴퓨터가 자연스럽게 이해한다.

WordPiece 실습

- 실습 개요

1) 텍스트 데이터: naver sentiment movie corpus(nsmc)

2) BertWordPieceTokenizer로 단어 사전 생성

① WordPiece 알고리즘을 구현하는 대표적인 Tokenizer

② 입력된 말뭉치(Corpus)에 존재하는 모든 개별 문자(Character/Alphabet)와 특수문자를 포함한 가장 작은 초기 단어 사전에서 시작

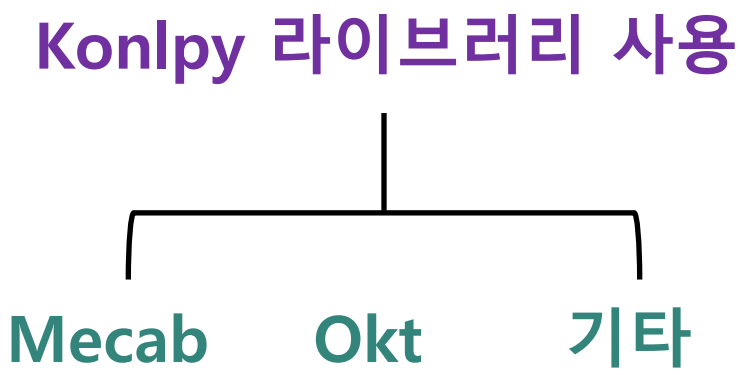
③ 입력된 말뭉치(Corpus)로부터 최적의 서브워드 사전(vocab.txt)을 생성해 내는 학습 전용 도구

3) BertTokenizerFast로 한글 문장 토큰화

: BertTokenizerFast → BertWordPieceTokenizer가 만든 vocab.txt 사전을 기반으로, 실제 BERT 언어 모델이 처리할 수 있는 완벽한 입력 형태를 만들어 주는 도구

WordPiece 실습

한글 텍스트 '형태소' 단위 토큰화(tokenization)



한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ Okt 형태소 분석기

- 라이브러리 설치 및 임폴트

1) 라이브러리 설치

```
pip install konlpy
```

2) 도구 임폴트

```
from konlpy.tag import Okt
```

- 클래스 함수 사용 방법

1) 클래스 호출, 형태소 분석 도구 생성

```
okt = Okt()
```

2) 기능1. 형태소 분석

① okt.morphs(text)

② 결과물 : 리스트 [형태소1, 형태소2, ...]

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ Okt 형태소 분석기

- 클래스 함수 사용 방법

4) 기능2. 형태소 분석 + 품사 tagging

① okt.pos(text)

② 결과물 : 리스트 [(형태소1, 품사), (형태소2, 품사2), ...]

5) 기능3. 명사 추출

① okt.nouns(text)

② 결과물 : 리스트 [명사1, 명사2, ...]

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ Okt 형태소 분석기 품사 정리

Noun	명사, 다음을 포함 - 명사, 대명사(Pronouns), - 회사와 이름 등 고유명사, - 수사(Numerals / 하나 둘 셋), - 독립 실행형 의존명사(Dependent)
Verb	동사
Adjective	형용사
Determiner	관형사 (참, 첫, 이, 그)
Adverb	부사 (매우, 빨리, 반드시)
Conjunction	접속사
Exclamation	감탄사
Josa	조사
PreEmoi	선어말어미 (있)
Emoi	어미 (요, 여)
Suffix	접미사

Punctuation	구두점, 다음을 모두 포함 - 마침표, 물음표, 느낌표, 줄임표, 따옴표, 괄호, 쉼표, 가운뎃점, 콜론, 빗금, 붙임표
Foreign	외국어, 한자 및 기타 기호
Alpha	알파벳
Number	숫자
KoreanParticle	한국어 자음, 모음 (ㅋㅋㅋ, ㅓㅜㅏ)
Hashtag	트위터 해시태그
ScreenName	트위터 아이디 (@neltia)
Email	이메일 주소
URL	웹 주소

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ Okt 형태소 분석기 실습

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ mecab 형태소 분석기

- 라이브러리 설치 및 임폴트

1) 라이브러리 설치

```
pip install python-mecab-ko
```

2) 도구 임폴트

```
from mecab import MeCab as Mecab
```

- 클래스 함수 사용 방법

1) 클래스 호출, 도구 생성

```
mecab = Mecab()
```

2) 기능1. 형태소 분석

① mecab.morphs(text)

② 결과물 : 리스트 [형태소1, 형태소2, ...]

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ mecab 형태소 분석기

- 클래스 함수 사용 방법

4) 기능2. 형태소 분석 + 품사 tagging

① mecab.pos(text)

② 결과물 : 리스트 [(형태소1, 품사), (형태소2, 품사2), ...]

5) 기능3. 명사 추출

① mecab.nouns(text)

② 결과물 : 리스트 [명사1, 명사2, ...]

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ mecab 형태소 분석기 품사 정리

대분류	세종 품사 태그	
	태그	설명
체언	NNG	일반 명사
	NNP	고유 명사
	NNB	의존 명사
	NR	수사
	NP	대명사
용언	VV	동사
	VA	형용사
	VX	보조 용언
	VCP	긍정 지정사
	VCN	부정 지정사
관형사	MM	관형사
부사	MAG	일반 부사
	MAJ	접속 부사
감탄사	IC	감탄사

조사	JKS	주격 조사
	JKC	보격 조사
	JKG	관형격 조사
	JKO	목적격 조사
	JKB	부사격 조사
	JKV	호격 조사
	JKQ	인용격 조사
	JX	보조사
	JC	접속 조사
선어말 어미	EP	선어말 어미
어말 어미	EF	종결 어미
	EC	연결 어미
	ETN	명사형 전성 어미
	ETM	관형형 전성 어미
접두사	XPN	체언 접두사
접미사	XSN	명사 파생 접미사
	XSV	동사 파생 접미사
	XSA	형용사 파생 접미사
어근	XR	어근

부호	SF	마침표, 물음표, 느낌표
	SE	줄임표
	SS	따옴표,괄호표,줄표
	SP	쉼표,가운뎃점,콜론,빗금
	SO	붙임표(물결,숨김,빠짐)
	SW	기타기호 (논리수학기호,화폐기호)
한글 이외	SL	외국어
	SH	한자
	SN	숫자

한글 텍스트 '형태소' 단위 토큰화(tokenization)

▶ mecab 형태소 분석기 실습

단어 빈도수 분석 및 시각화 실습

- 실습 개요

  1) 한글 텍스트 데이터 → 한글 형태소 분석기 → 형태소 단위로 토큰화 → 명사(Noun) 추출

  2) python collections 패키지 → Counter() 이용 → 빈도수 분석

  3) 빈도수 분석 결과 → wordcloud 라이브러리를 이용하여 시각화



mecab 형태소 분석기

단어 빈도수 분석 및 시각화 실습

- python collections 패키지 Counter() 함수

1) 기능 : 문자열, 리스트, 튜플 등 여러 성분 원소를 저장하고 있는 자료형에서 각 성분 원소의 빈도수를 계산해주는 함수

2) 예시

```
# 필요한 함수 임폴트
from collections import Counter


# 함수 실행의 결과
Counter(["hi", "hey", "hi", "hi", "hello", "hey"])
>>> Counter({'hi': 3, 'hey': 2, 'hello': 1})
```

단어 빈도수 분석 및 시각화 실습

- wordcloud 라이브러리

1) 텍스트에 등장하는 단어들의 빈도수를 기반으로 텍스트 데이터의 주요 단어들을 시각적으로 표현하는데 사용되는 라이브러리

2) 사용 방법

① 필요한 라이브러리 / 함수 임폴트

② 한글 폰트 설정

③ WordClod 클래스 함수 객체 생성

④ wordcloud 이미지 생성

⑤ 이미지 저장

⑥ 이미지 출력

단어 빈도수 분석 및 시각화 실습

- 대한민국 헌법

단어 빈도수 분석 및 시각화 실습

- 마켓컬리 사미헌 갈비탕 리뷰 데이터(1page ~ 50page 수집 후 진행)

21. 자연어 딥러닝

임베딩(Embedding)



- 정의 : 텍스트를 숫자로 표현
  - ① 방법 : 단어를 숫자로 표현
  - ② 표현의 형식
    - 희소 표현(sparse representation)
    - 밀집 표현(dense representation)

임베딩(Embedding)

희소 표현(sparse representation)

- 대부분이 0으로 표현되는 방법

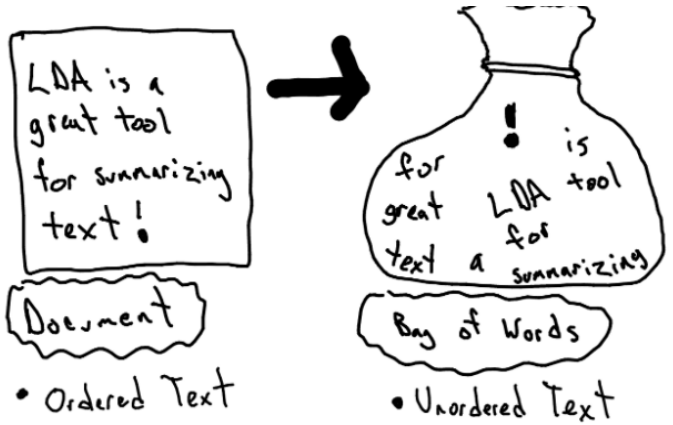
  예) 나는 배가 고프다 → [1 0 0 1 0 0 0 1 0 0]

- BoW(Bag of Words)

  ① 단어들의 문맥이나 순서를 무시

  ② 단어들의 출현 빈도수를 카운트(count)하여 단어를 수치화, 문장을 수치화하는 방법

  ③ 모델 : CountVectorizer, TfidfVectorizer



임베딩(Embedding)

CountVectorizer 모델을 이용한 BoW(Bag of Words) 구현

- 구현 방법

① 텍스트 데이터(말뭉치, corpus) → 토큰화 → 단어 사전{'단어':인덱스} 생성

② 임베딩 할 문장(sentence) : 단어 사전에 있는 각 단어들의 출현 빈도수로 표현

CountVectorizer 모델을 이용한 BoW 구현 예시

단어사전

{'고프다':0, '공부':1, '공부해야지': 2, '나는':3, '내일':4, '먹고': 5, '뭐먹지': 6, '배가': 7, '점심': 8, '해야겠다': 9}



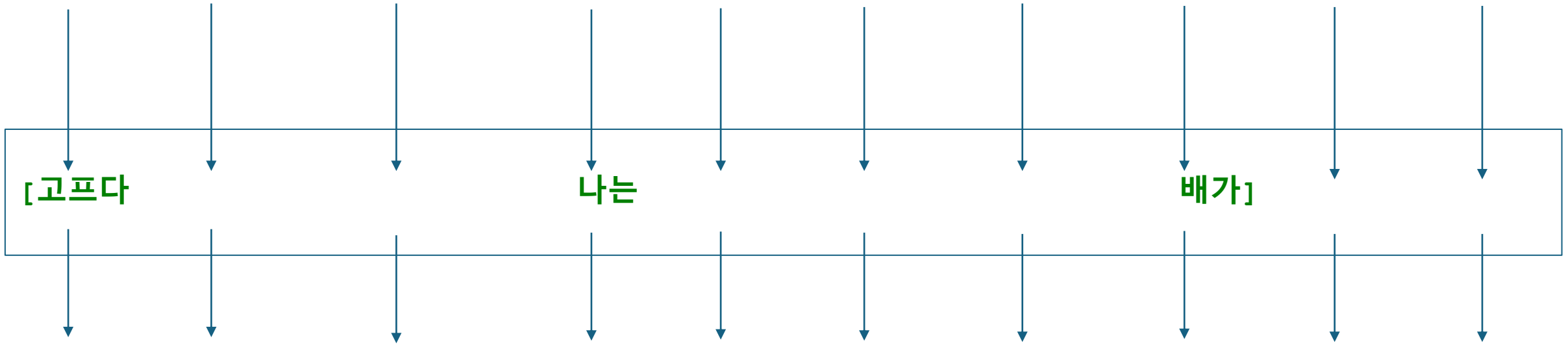
Sentence = ['나는 배가 고프다']



[[1 0 0 1 0 0 0 1 0 0]]

단어사전(기준)

'고프다':0, '공부':1, '공부해야지': 2, '나는':3, '내일':4, '먹고': 5, '뭐먹지': 6, '배가': 7, '점심': 8, '해야겠다': 9



1번	0번	0번	1번	0번	0번	0번	1번	0번	0번
1	0	0	1	0	0	0	1	0	0

[[1 0 0 1 0 0 0 1 0 0]]

임베딩(Embedding) : CountVectorizer 실습

1. 필요한 함수 임포트

   ```
   from sklearn.feature_extraction.text import CountVectorizer
   ```

2. CountVectorizer 호출, 임베딩 모델 생성

3. fit() 함수 호출 → 토큰화(어절 단위) → 토큰별 단어 사전 생성

4. transform([text1, ...]).toarray() 함수 호출 → 임베딩 행렬 생성

임베딩(Embedding) : CountVectorizer 실습

임베딩(Embedding)

TfidfVectorizer 모델을 이용한 BoW(Bag of Words) 구현

- 단순 빈도수(Count) 기반 수치화의 한계 극복
  1) 모든 문서(문장)에 공통적으로 자주 등장하는 단어(예: 조사 등)의 정보 가치를 낮춤
  2) 특정 문서(문장)에만 집중적으로 등장하는 중요한 핵심 단어의 가중치를 높여 수치화
- 기본 개념
  1) TF (Term Frequency): 특정 문서 내 단어의 출현 빈도 (자주 나올수록 중요)
  2) IDF (Inverse Document Frequency)
     ① 단어가 등장한 문서 수의 역수 ($\frac{N}{DF}$)
     → N: 말뭉치 내 전체 문서 수, DF: 단어 t가 출현한 문서 수 (Document Frequency)
     ② 특정 단어가 여러 문서에 자주 등장할수록, 불용어일 확률이 크므로 해당 단어의 중요도를 낮추기 위해 작은 값을 부여하는 개념
  3) TF-IDF : TF와 IDF을 곱한 값, 문서 전체에서 단어의 중요도를 나타내는 값(숫자)

임베딩(Embedding)

TfidfVectorizer 모델을 이용한 BoW(Bag of Words) 구현

- 단어별 등장 문서 수(Document Frequency) 집계

문장 1: '나는 배가 고프다'
문장 2: '내일 점심 뭐먹지'
문장 3: '내일 공부 해야겠다'
문장 4: '점심 먹고 공부해야지'

단어 (Token)	등장한 문장 ID	df(t)
내일	문장 2, 문장 3	2
점심	문장 2, 문장 4	2
나는	문장 1	1
배가	문장 1	1
고프다	문장 1	1
뭐먹지	문장 2	1
공부	문장 3	1
해야겠다	문장 3	1
먹고	문장 4	1
공부해야지	문장 4	1

임베딩(Embedding)

TfidfVectorizer 모델을 이용한 BoW(Bag of Words) 구현

- TfidfVectorizer 가중치 IDF 계산 공식

$$\mathrm{IDF}(t) = \ln\left(\frac{1+N}{1+\mathrm{df}(t)}\right)+1$$

1) N: 말뭉치 내 전체 문서(문장) 수
2) df(t): 단어 t가 출현한 문서 수 (Document Frequency)
3) TF-IDF = TF × IDF

임베딩(Embedding) : TfidfVectorizer 실습

1. 필요한 함수 임포트

   from sklearn.feature_extraction.text import TfidfVectorizer

2. TfidfVectorizer 호출, 임베딩 모델 생성

3. fit() 함수 호출 → 토큰화(어절 단위) → 토큰별 단어 사전 생성, IDF 가중치 학습

4. transform([text1, ...]).toarray() 함수 호출 → TF-IDF 임베딩 행렬 생성

임베딩(Embedding) : TfidfVectorizer 실습

임베딩(Embedding)

텍스트 유사도(Similarity)

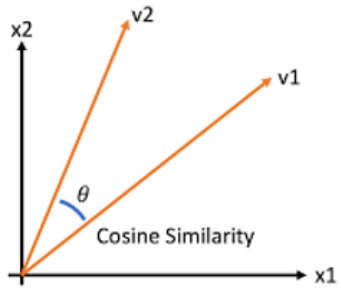
- 개념 : 두 문장이 주어졌을 때, 두 문장이 얼마나 비슷한 지를 측정
  1) 예시) 고객응대 챗봇
     A : "이 요리의 레시피를 알려줘."
     B : "이 요리 어떻게 만드는지 알려줘."
  2) 두 문장은 같은 의미이므로 같은 대답을 하도록 개발
  3) 유사도 측정 기준 : 코사인 유사도

임베딩(Embedding)

코사인 유사도(Cosine Similarity)

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} (A_i)^2} \times \sqrt{\sum_{i=1}^{n} (B_i)^2}}$$

- 벡터 사이의 각도를 측정해서 두 벡터가 얼마나 유사한지 측정
- 두 벡터 사이의 각도 : 0 ~ 180





코사인 유사도 1

각도 = 0

유사도 최대



코사인 유사도 0

각도 = 90

관계 없음



코사인 유사도 -1

각도 = 180

정반대

임베딩(Embedding)

코사인 유사도(Cosine Similarity) 구현

- sklearn.metrics.pairwise.cosine_similarity 사용

  1) 필요한 라이브러리 임폴트

  ```
  from sklearn.metrics.pairwise import cosine_similarity
  ```

  2) similarity = cosine_similarity (X, Y)

- 적용 : 유사도 기반 챗봇 개발, 유사도 기반 추천 함수 구현

임베딩(Embedding)

CountVectorizer와 코사인 유사도를 이용한 텍스트 유사도 측정 실습

임베딩(Embedding)

TfidfVectorizer와 코사인 유사도를 이용한 텍스트 유사도 측정 실습

임베딩(Embedding)

CountVectorizer와 코사인 유사도를 이용한 추천 시스템 구현 실습

- 실습 개요

1) dataset
   ① movies_metadata.csv
   ② 2017년 7월 또는 그 이전에 개봉된 영화 45,466편에 대한 각종 정보가 포함
2) 영화 소개(줄거리, 텍스트 데이터)를 CountVectorizer를 이용하여 행렬로 변환
3) 좋아하는 영화의 제목을 입력
4) 입력한 영화의 줄거리와 유사한 줄거리를 가지는 영화 제목을 찾아서 추천

임베딩(Embedding)

CountVectorizer와 코사인 유사도를 이용한 추천 시스템 구현 실습

임베딩(Embedding)

TfidfVectorizer와 코사인 유사도를 이용한 추천 시스템 구현 실습

- 실습 개요

1) dataset
   ① movies_metadata.csv
   ② 2017년 7월 또는 그 이전에 개봉된 영화 45,466편에 대한 각종 정보가 포함
2) 영화 소개(줄거리, 텍스트 데이터)를 TfidfVectorizer를 이용하여 행렬로 변환
3) 좋아하는 영화의 제목을 입력
4) 입력한 영화의 줄거리와 유사한 줄거리를 가지는 영화 제목을 찾아서 추천

임베딩(Embedding)

CountVectorizer와 코사인 유사도를 이용한 추천 시스템 구현 실습

임베딩(Embedding)

밀집 표현(dense representation)

- 학습을 통해서 토큰(단어)나 문장을 0이 아닌 일정한 크기의 실수로 표현하는 방법

삽살개	[0.11 0.8 0.52 0.76 0.91]
진돗개	[0.33 0.1 0.56 0.79 0.71]
고양이	[0.91 0.35 0.51 0.88 0.1]
돌고래	[0.41 0.27 0.43 0.13 0.9]
원숭이	[0.44 0.23 0.91 0.89 0.2]
•	•
•	•
•	•

```
[-4.99942362e-01 -3.59200597e-01 -7.38401562e-02  4.84219134e-01
 -4.96614248e-01  5.82403481e-01 -4.56285924e-02 -3.81824046e-01
 -3.94545138e-01  2.18352944e-01  2.40236863e-01 -6.53119981e-01
  1.17322311e-01 -4.41113621e-01 -8.18817504e-03 -2.16763169e-01
  1.42986374e-02 -9.46456194e-02  1.55718774e-01 -1.03794351e-01
 -5.95203400e-01 -5.41155457e-01 -6.22601509e-01 -1.00601888e+00
 -2.20479354e-01 -1.32809699e-01 -4.36403155e-02 -4.01162505e-02
  5.56914866e-01  7.56343603e-01  6.72921762e-02  8.65490198e-01
  4.12386581e-02  2.79036552e-01 -1.98162034e-01 -5.00127748e-02
  3.10566962e-01 -3.46614659e-01 -6.20578766e-01  6.81772888e-01
  8.81858394e-02  2.54626304e-01 -4.60292935e-01 -1.45168126e-01]
```

- LLM(거대 언어 모델)이나 생성형 AI와 같은 언어 모델의 발전 과정

  : 사람이 텍스트를 이해하는 방식과 유사하게 전체 문맥, 뉘앙스, 동음이의어 등의 정보를 풍부하게 반영하도록 텍스트 데이터에 대한 임베딩(Embedding) 성능을 향상시키는 과정

학습 : 모델 적용 및 결과 도출



순환 신경망 - RNN (Recurrent Neural Network)

▶ 대상이 되는 데이터 : 순차 데이터(Sequential Data)

- 시간적, 순차적(순서) 특성을 가진 데이터

- 순서가 변경될 경우 고유의 특성을 잃어버리는 특징이 있음, 한가지 예로, '도'와 '시'라는 글자로 '도시'와 '시도'라는 단어들의 조합이 가능하지만 이들은 서로 다른 의미를 갖는 것과 같음

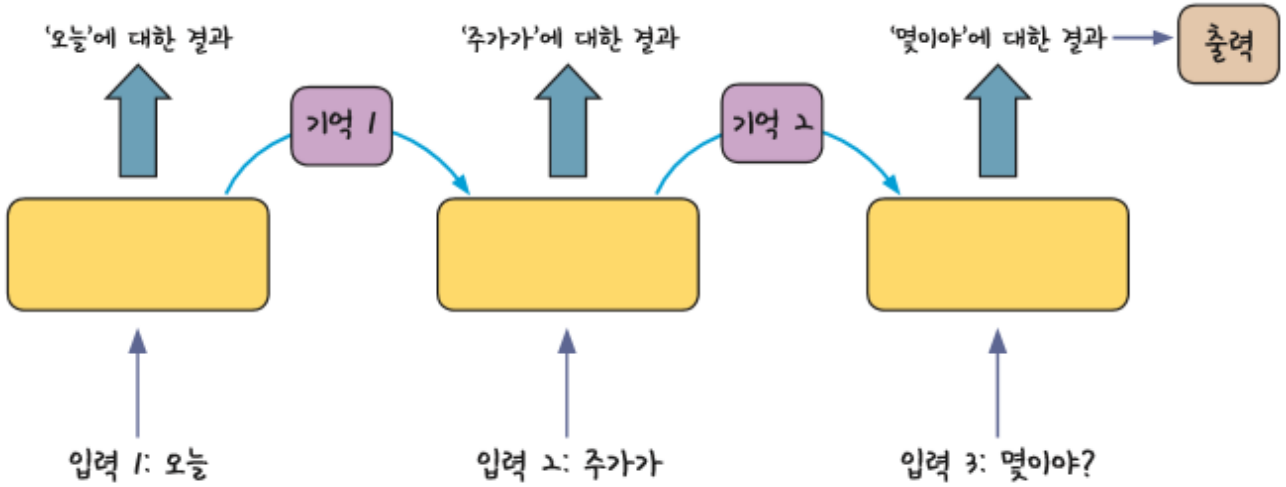
- 종류 : 텍스트 데이터, 시계열 데이터(주가 및 환율 변동, 제품 수요 예측 등)

순환 신경망 - RNN (Recurrent Neural Network)

▶ RNN의 기본 원리

- 과거의 기억을 저장 : 여러 개의 데이터가 순서대로 입력되었을 때 앞서 입력 받은 데이터의 연산 결과를 기억(저장)한다.

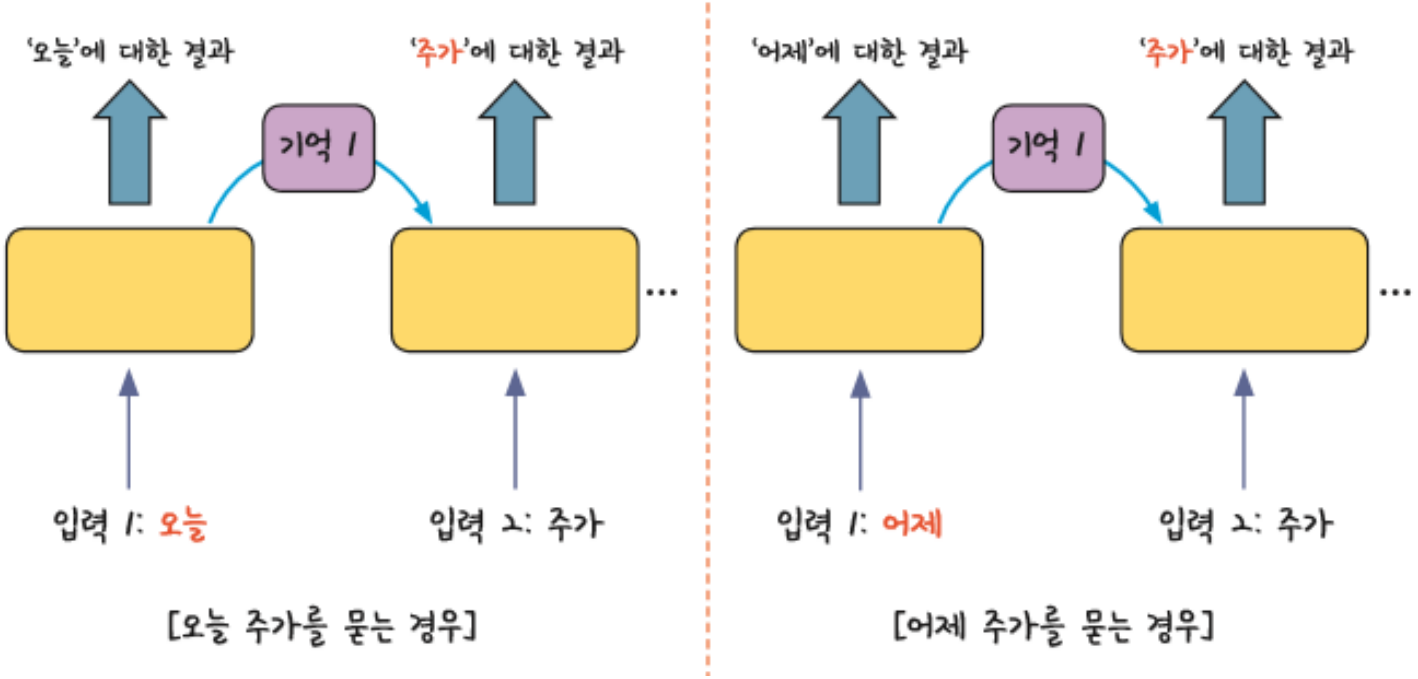
- 과거의 기억을 현재 학습에 반영 : 기억된 데이터(과거)를 가지고 현재 입력과 함께 연산한다.



순환 신경망 - RNN (Recurrent Neural Network)

▶ 다른 신경망과의 차이

- 과거의 결과가 현재 입력 값에 영향을 준다.
- 비슷한 두 문장이 입력되어도 과거의 차이로 인해서 두 문장의 차이를 반영할 수 있다.
- 반복적이고 순차적인 데이터(Sequential) 학습에 적합한 인공신경망



순환 신경망 - RNN (Recurrent Neural Network)

▶ RNN을 이용한 실습: IMDB 영화 리뷰 감성 분석 실습

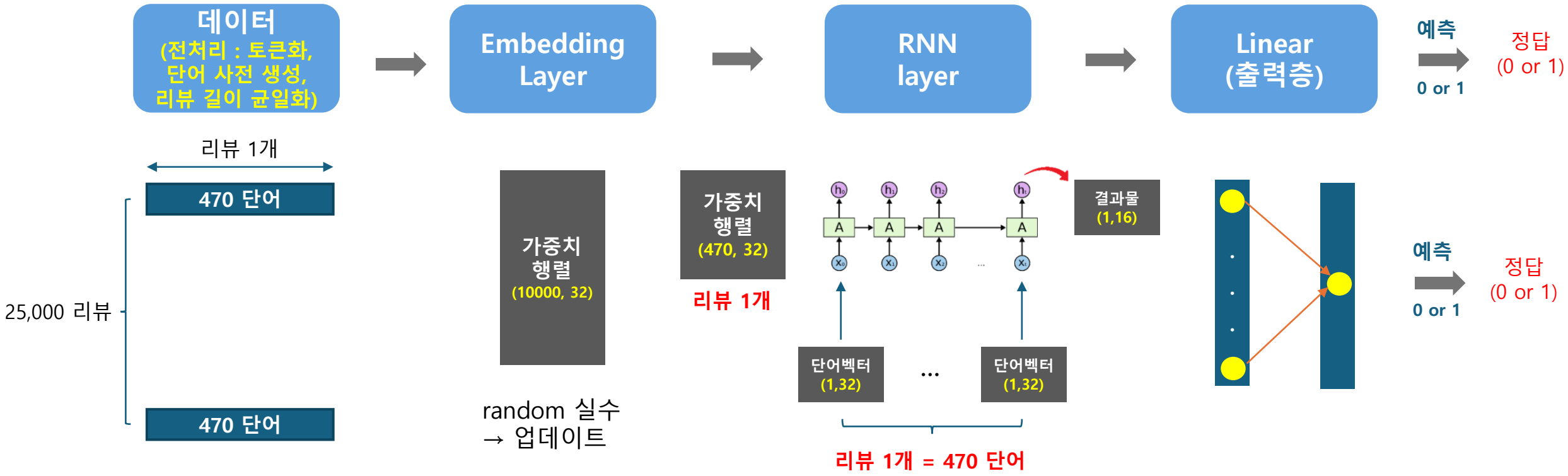
- 실습 개요

1) 데이터 : 50,000개의 영화 리뷰와 감성(sentiment, 긍정/부정)으로 구성된 IMDB
2) 데이터 → 학습용 데이터 25,000개 + 평가용 데이터 25,000개
3) 목표 : review의 문장(단어 sequence)을 기반으로 review의 sentiment를 예측하는 RNN 모델 생성, 학습, 평가 진행

1	review	sentiment
	One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happened with me. The first thing that struck me about Oz was its brutality and unflinching scenes of violence, which set in right from the word GO. Trust me, this is not a show for the faint hearted or timid. This show pulls no punches with regards to drugs, sex or violence. Its is hardcore, in the classic use of the word. It is called OZ as that is the nickname given to the Oswald Maximum Security State Penitentary. It focuses mainly on Emerald City, an experimental section of the prison where all the cells have glass fronts and face inwards, so privacy is not high on the agenda. Em City is home to many..Aryans, Muslims, gangstas, Latinos, Christians, Italians, Irish and more....so scuffles, death stares, dodgy dealings and shady agreements are never far away. I would say the main appeal of the show is due to the fact that it goes where other shows wouldn't dare. Forget pretty pictures painted for mainstream audiences, forget charm, forget romance...OZ doesn't mess around. The first episode I ever saw struck me as so nasty it was surreal, I couldn't say I was ready for it, but as I watched more, I developed a taste for Oz,	1

순환 신경망 - RNN (Recurrent Neural Network)

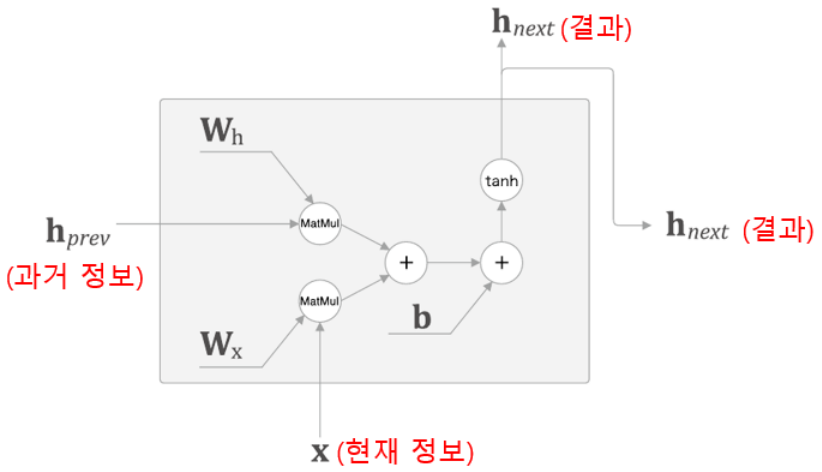
- 영화 리뷰 감성 분석 모델 구성과 데이터의 흐름



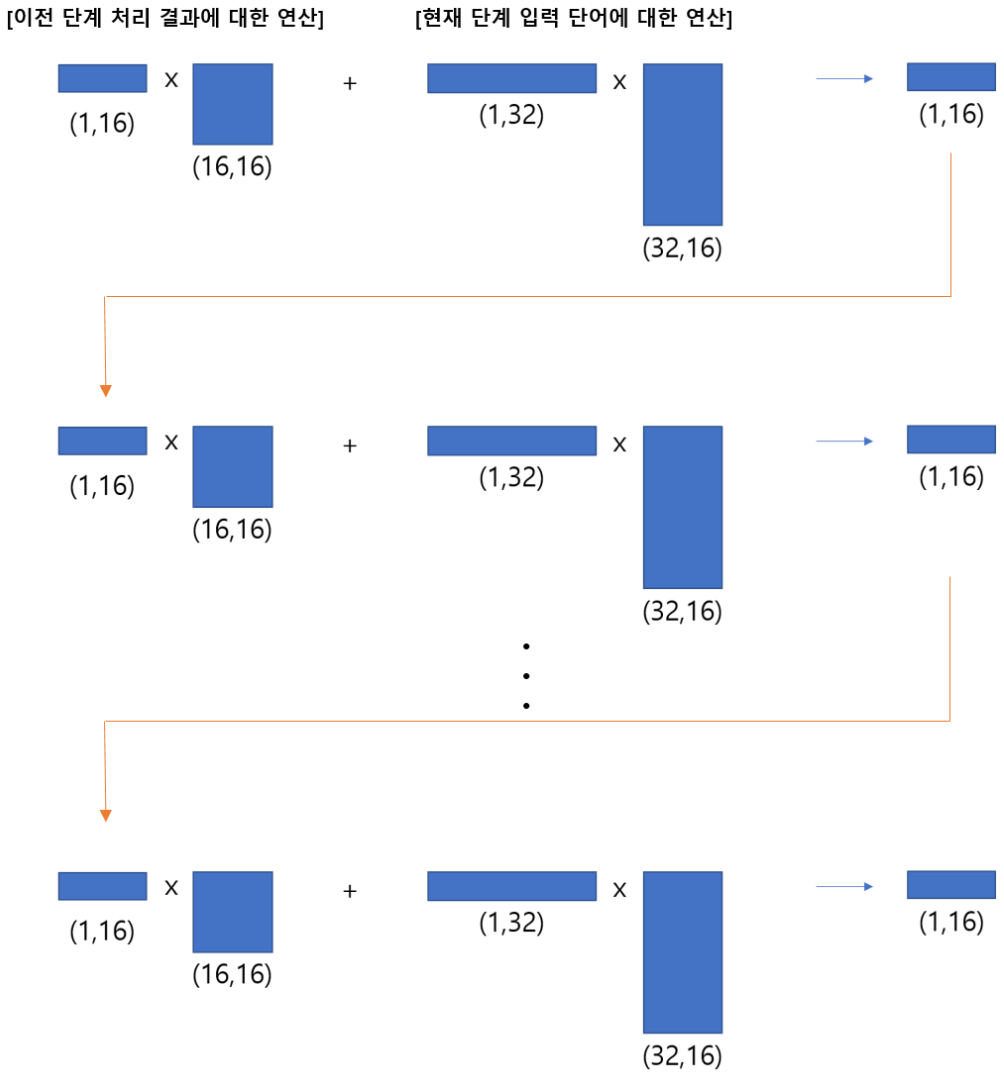
text data → 단어 : 임베딩 벡터 → 입력 문장(문단) : 임베딩 벡터 → 전체 text data에 대한 임베딩 행렬(특성 추출) → 예측(분류)

순환 신경망 - RNN (Recurrent Neural Network)

- RNN 모델
  1) 입력 : 단어 임베딩 행렬
  2) 출력 : 문장 임베딩 행렬
  3) 기본 구조

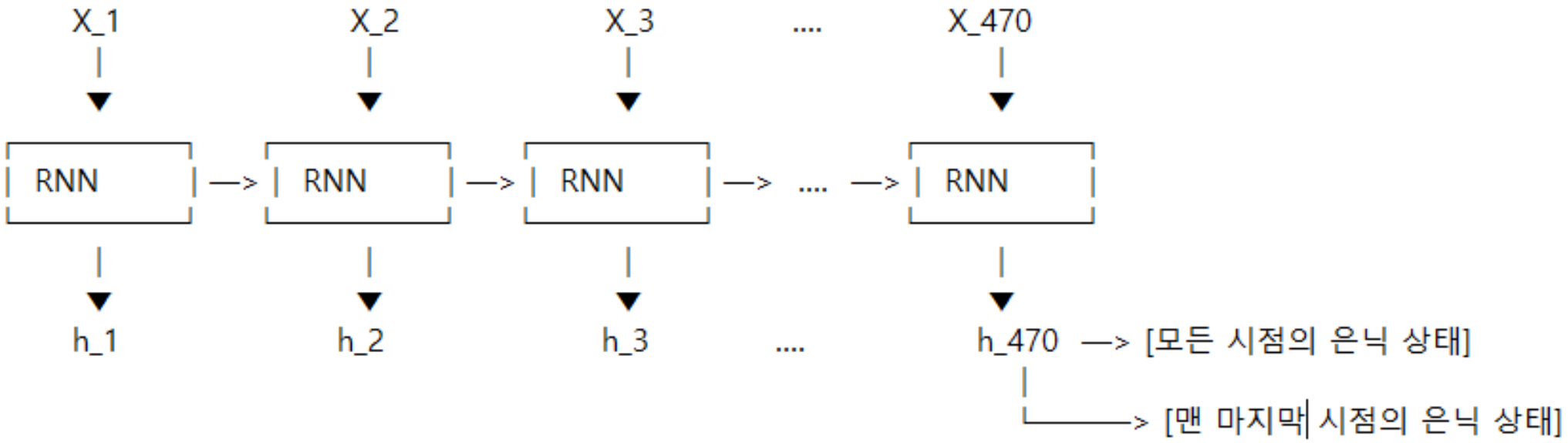


$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$



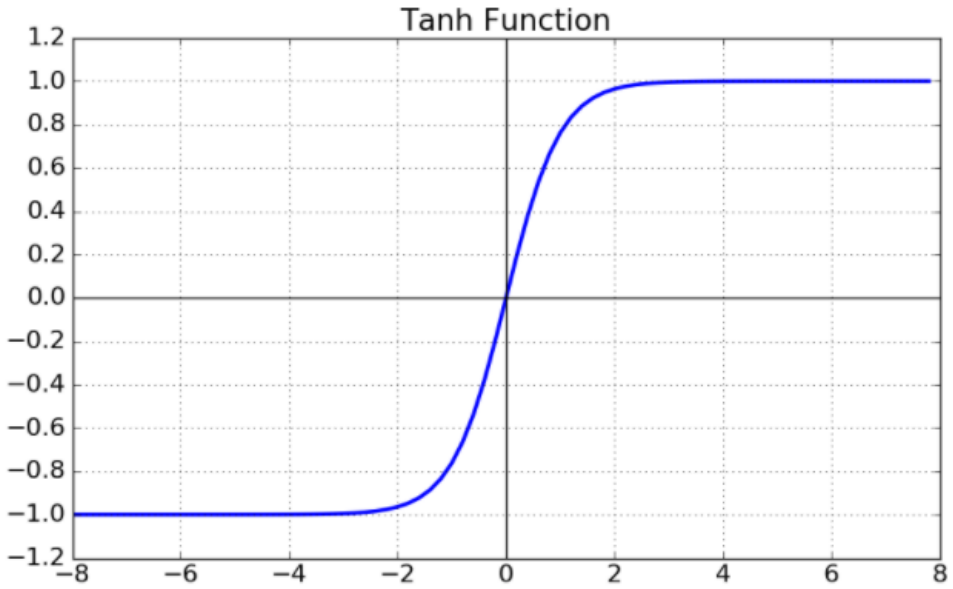
순환 신경망 - RNN (Recurrent Neural Network)

- RNN 모델 연산 과정



순환 신경망 - RNN (Recurrent Neural Network)

- 활성화 함수
  - tanh(하이퍼블릭 탄젠트) 함수 : 시그모이드 함수와 비슷, 출력 값이 -1과 1사이에 존재



순환 신경망 - RNN (Recurrent Neural Network)

- RNN의 학습 원리 : BPTT (Backpropagation Through Time)

1) BPTT (Time-step에 따른 오차 역전파)

① 앞서 배운 역전파 알고리즘(Chain Rule)을 시퀀스 데이터 처리를 위해 시간에 따라 진행되는 RNN 구조에 적용

② 마지막 시점(t)의 오차(Loss)로부터 과거 시점(t-1, t-2, ..., 1) 방향으로 거꾸로 기울기를 전파하며 가중치 매개변수 (W_h, W_x, b) 업데이트

2) BPTT의 기본 동작 흐름

① 1단계 (Forward Pass) : 토큰 입력 순서대로(t=1부터 t=T까지) 입력 데이터를 연산하며 은닉 상태(h_t)와 최종 출력값 (확률) 계산

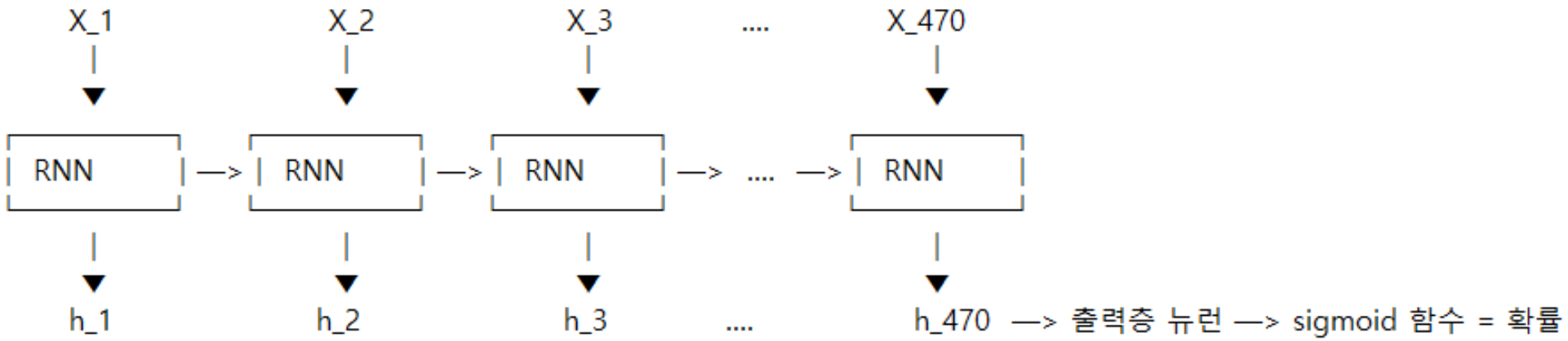
② 2단계 (Loss 계산) : 최종 출력값(확률) 과 정답(Label) 간의 오차(Loss) 계산

③ 3단계 (Backward Pass) : 시간의 역순(t=T부터 t=1까지)으로 연쇄법칙을 적용하여 가중치 기울기 계산 및 업데이트

순환 신경망 - RNN (Recurrent Neural Network)

- RNN의 학습 원리 : BPTT (Backpropagation Through Time)

[순방향 연산]



순환 신경망 - RNN (Recurrent Neural Network)

- RNN의 학습 원리 : BPTT (Backpropagation Through Time)

[오차 역전파: t=1에서 입력 데이터 X의 가중치 W_x의 손실에 대한 기울기]

$$\frac{\partial L}{\partial W_x(t=1)} = \frac{\partial L}{\partial h_T} \times \frac{\partial h_T}{\partial h_{T-1}} \times \frac{\partial h_{T-1}}{\partial h_{T-2}} \times \ldots \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial W_x}$$

순환 신경망 - RNN (Recurrent Neural Network)

- 기본 RNN의 한계 (1): 기울기 소멸 (Vanishing Gradient)

1) 기울기 소멸(Vanishing Gradient) 현상

① 역전파 진행 시 연쇄 법칙(Chain Rule)에 의해 활성화 함수(tanh)의 미분값이 시간 축을 따라 계속해서 반복 곱해 짐

② tanh의 미분값이 1 이하의 실수이기 때문에 1보다 작은 tanh의 미분값을 곱하면 값이 줄어들게 되는데, 이를 역전파 과정 내내 반복해서 곱하고 전달하다 보니 결국 기울기(Gradient)가 0으로 수렴하게 됨

$$\frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial h_k}{\partial z_k} \times \frac{\partial z_k}{\partial h_{k-1}}$$

$$\text{where, } h_k = \tanh(z_k), \quad z_k = W_h h_{k-1} + W_x x_k + b_h$$

$\frac{\partial h_k}{\partial z_k}$ ⟶ tanh 미분값 <= 1

순환 신경망 - RNN (Recurrent Neural Network)

- 기본 RNN의 한계 (1): 기울기 소멸 (Vanishing Gradient)

2) 기울기 소멸이 모델 학습에 미치는 영향

① 문장 길이(Sequence Length)가 길어질수록 문장 초반(t = 1, 2, ...) 단어들의 기울기가 0에 수렴

② 초반 가중치 업데이트가 중단되어 문장 앞부분의 의미를 학습할 수 없는 상태 발생

3) 기울기 전달 양상 (예시)



순환 신경망 - RNN (Recurrent Neural Network)

- 기본 RNN의 한계 (2): 장기 의존성 문제 및 병렬 처리의 한계

1) 장기 의존성 문제 (Long-Term Dependency Problem)

① 원인 : 기울기 소멸 현상으로 인해 먼 과거의 정보가 현재 시점까지 전달되지 못함

② 현상 : 입력 토큰의 길이가 길어질수록 문장 초반의 정보가 희석되거나 사라지는 현상 발생

→ 긴 문장에 대한 예측 정확도 감소

2) 구조적 한계 : 순차적 연산으로 인한 병렬 처리 불가

① t 시점의 은닉 상태(h_t) 계산을 위해 반드시 이전 시점(h_{t-1}) 계산이 완료되어야 함

② GPU의 대규모 병렬 연산(Parallel Processing)을 활용할 수 없어 거대 데이터 학습 시 속도 한계 발생

순환 신경망 - RNN (Recurrent Neural Network)

▶ RNN을 이용한 실습: IMDB 영화 리뷰 감성 분석 실습

- RNN 기반 분류 모델 클래스 구성

1) nn.Embedding : 단어 인덱스 정수 배열을 32차원 임베딩 행렬로 변환
2) nn.RNN : 이전 시점(t-1)의 정보와 현재(t) 단어 임베딩을 결합하여 문장 임베딩 연산
3) nn.Linear : 은닉층의 출력값을 받아 긍정/부정(1/0) 예측 결과 출력

순환 신경망 - RNN (Recurrent Neural Network)

▶ RNN을 이용한 실습: IMDB 영화 리뷰 감성 분석 실습

- RNN 기반 분류 모델 클래스 정의 코드

```
import torch
import torch.nn as nn

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size=10000, embed_dim=32, hidden_dim=16, output_dim=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x: (batch_size, 470)
        embedded = self.embedding(x)          # (batch_size, 470) -> (batch_size, 470, 32)
        output, hidden = self.rnn(embedded)   # hidden: (1, batch_size, 16)
        out = self.fc(hidden.squeeze(0))      # (batch_size, 16) -> (batch_size, 1)
        return self.sigmoid(out)
```

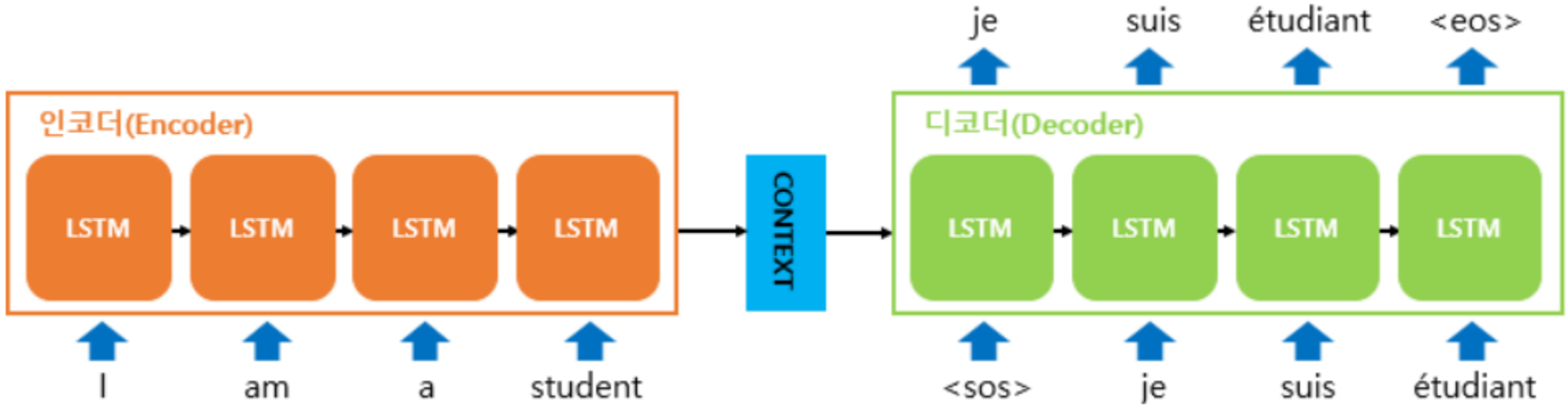
순환 신경망 - RNN (Recurrent Neural Network)

▶ RNN을 이용한 실습: IMDB 영화 리뷰 감성 분석 실습

seq2seq 모델

- 두 개의 순환 신경망(recurrent neural network)로 이루어진 학습모델
  1) encoder: 입력 값을 받아 이를 고정된 크기의 문맥 벡터(context vector)로 변환(읽기)
  2) decoder: 문맥 벡터를 사용해서 출력 값 생성(쓰기)

- 하나의 텍스트 문장이 입력으로 들어오면 하나의 텍스트 문장을 출력하는 구조, 생성 모델, 기계 번역에서 사용되는 모델



Transformer

- 순환 신경망의 한계

  1) 기울기 소멸 및 장기 의존성 문제 (앞부분 단어 정보 손실)

  2) 입력 시퀀스의 순서에 따라 순차적으로 연산을 수행하기 때문에 병렬 처리가 어려움. 이는 대규모 데이터를 다루기 어렵게 만듦

- Transformer

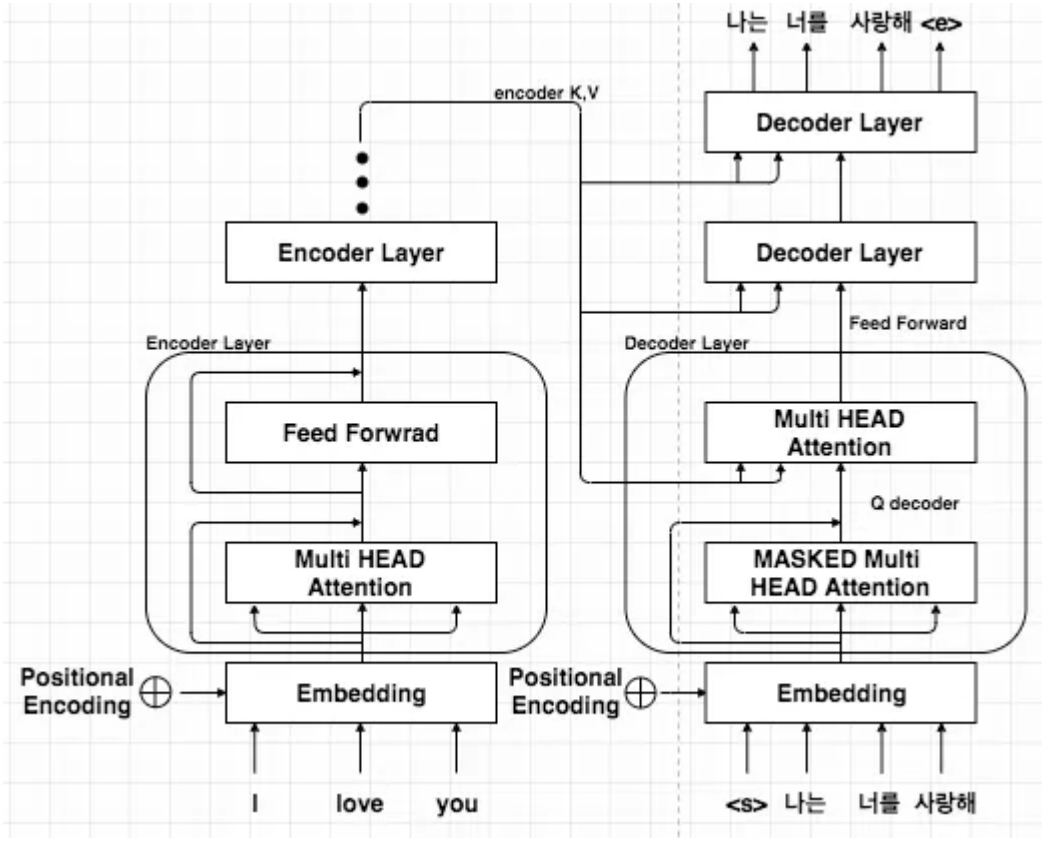
  1) 2017년 구글이 발표한 논문(Attention is all you need)

  2) 순환 신경망의 한계를 극복하기 위해 제안된 모델

  3) RNN을 사용하지 않고, 어텐션(attention) 알고리즘을 통해 시퀀스 데이터를 처리하는 모델

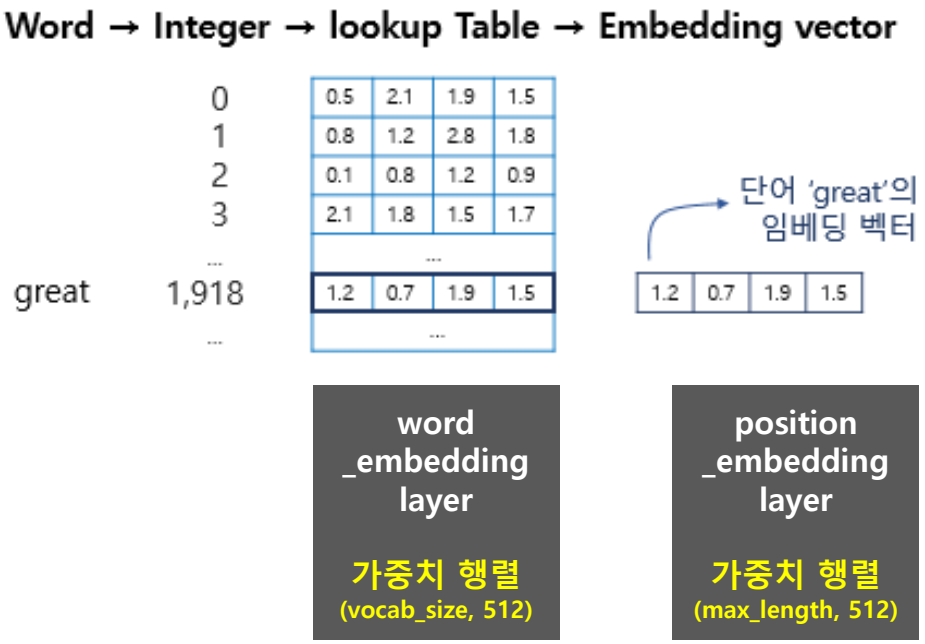
Transformer

- Transformer 모델의 구조 : 인코더(encoder) + 디코더(decoder)



Transformer

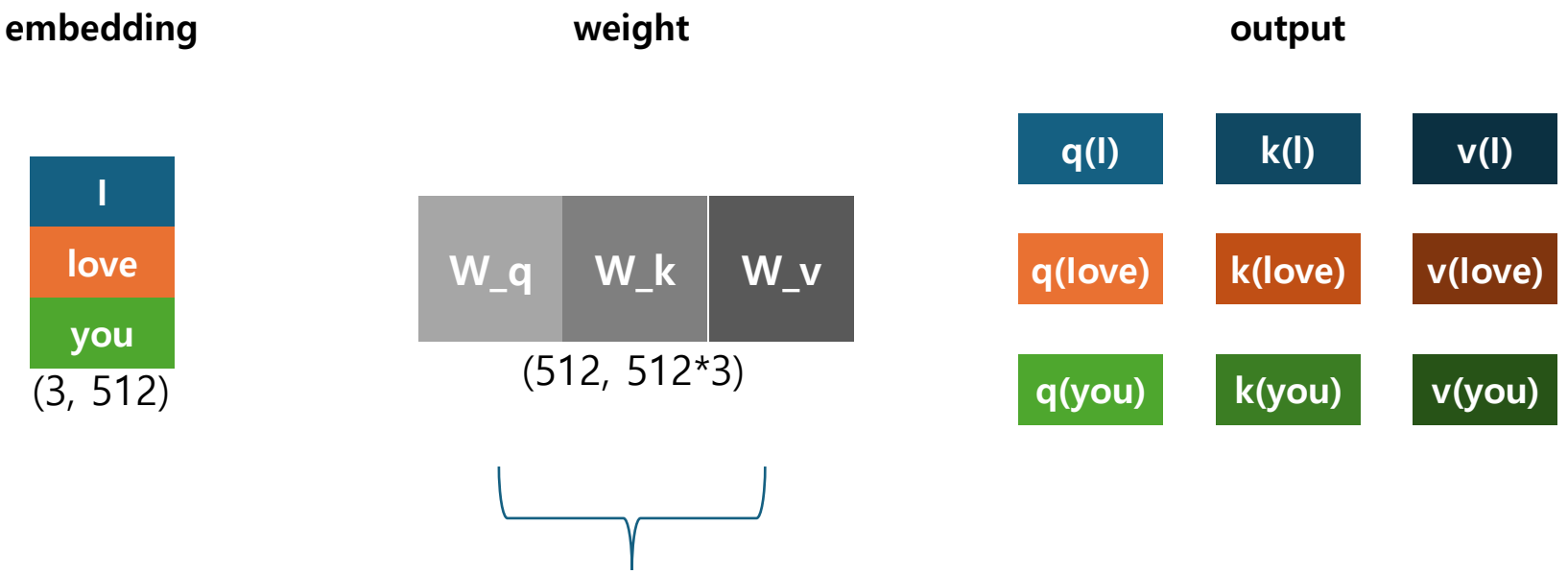
- self-attention
  - 입력 문장의 각 단어가 다른 단어들과 얼마나 관련이 있는지를 계산하고, 그 결과를 가중합하여 문장 전체의 문맥을 반영하는 contextual embedding vector를 생성하는 알고리즘
  - 과정1 : 입력 문장 → 토큰화(WordPiece tokenizer, 정수 인코딩) + 단어 임베딩 + positional encoding





Transformer

- self-attention
  - 과정2 : 각 단어 임베딩 벡터 → 쿼리(query), 키(key), 값(value) 벡터 생성
  - 쿼리(query), 키(key), 값(value) 벡터 간의 연산을 통해서 문맥적 관계성을 추출하는 과정



task(감성 분석 등)를 가장 잘 수행하는 방향으로
학습되는 과정에서 업데이트 됨

Transformer

- self-attention
  - 쿼리(query), 키(key), 값(value)의 역할

  1) query : 문장 내 다른 단어들과 어떤 관계를 가지는지를 파악하고자 하는 특정 단어의 벡터

  2) key : query가 문장 어떤 단어와 유사도가 큰 지를 계산하는 데 사용되는 단어 벡터

  → 입력 문장의 전체 단어 벡터

  3) value : 가중치를 적용하여 최종 출력을 생성하는 데 사용되는 단어 벡터

  → 입력 문장의 전체 단어 벡터

Transformer

- self-attention
  - 과정3 : Scaled Dot-Product Attention

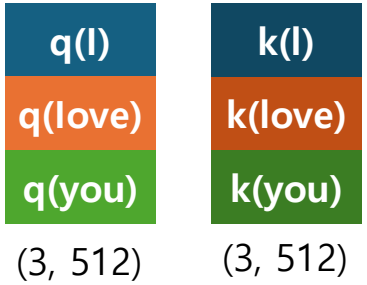
    1) attention score → query 벡터와 key 벡터의 내적을 통해 각 단어 간의 유사도 점수를 계산

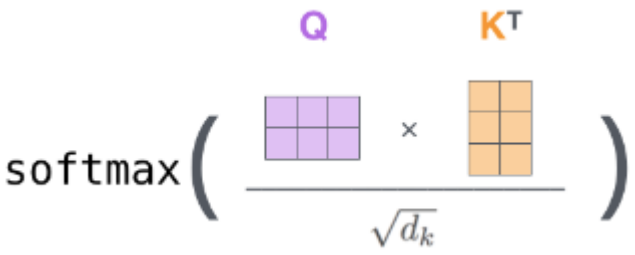
    2) scaling : attention score 값 / key 벡터의 차원 크기의 제곱근 → attention score 값의 크기를 조정

    3) attention weight

    ① 스케일링을 거친 attention score를 softmax 함수에 입력

    ② 입력 문장의 각 단어가 다른 단어들과 얼마나 관련이 있는지를 0과 1사이의 숫자로 표현





key

query

	I	love	you
I	0.9	0.03	0.07
love	0.05	0.85	0.1
you	0.06	0.04	0.9

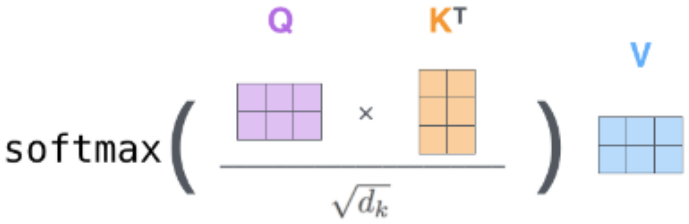
Transformer

- self-attention
  - 과정3 : Scaled Dot-Product Attention

    4) 어텐션 출력(attention output) 벡터 생성

    ① 어텐션 가중치(attention weight)와 v 벡터의 가중합 계산

    ② **문장을 구성하고 있는 각 단어 간의 관계를 반영한 새로운 단어 임베딩 벡터 생성**

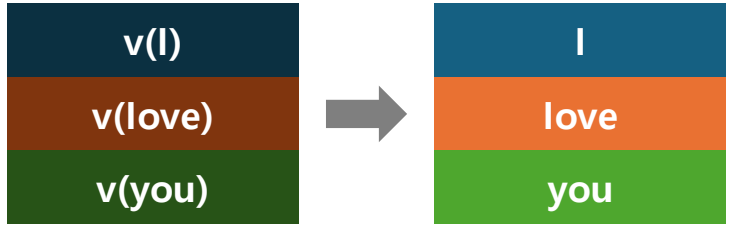


	I	love	you
I	0.9	0.03	0.07
love	0.05	0.85	0.1
you	0.06	0.04	0.9

[attention weight]

[각 단어 별 새로운 임베딩 벡터 생성]

[0.9 0.03 0.07]
[0.05 0.85 0.1]
[0.06 0.04 0.9]
(3, 3)



(3, 512) (3, 512)

Transformer

- self-attention
  - 과정3 : Scaled Dot-Product Attention

    4) 어텐션 출력(attention output) 벡터 생성 예시

0.9	0.03	0.07
0.05	0.85	0.1
0.06	0.04	0.9

[attention weight(3, 3)]

X

0.1	0.1	0.1	•••	0.1
0.2	0.2	0.2	•••	0.2
0.5	0.5	0.5	•••	0.5

[value matrix(3, 512)]

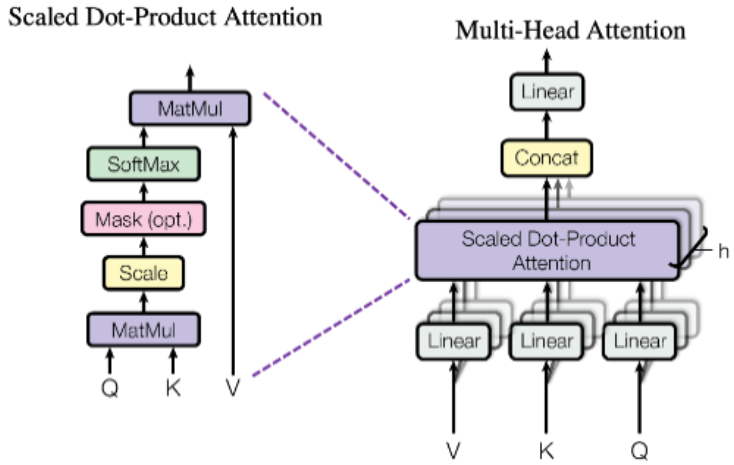
→

(0.9X0.1) + (0.03X0.2) + (0.07X0.5)	(0.9X0.1) + (0.03X0.2) + (0.07X0.5)	•••	(0.9X0.1) + (0.03X0.2) + (0.07X0.5)
(0.05X0.1) + (0.85X0.2) + (0.1X0.5)	(0.05X0.1) + (0.85X0.2) + (0.1X0.5)	•••	(0.05X0.1) + (0.85X0.2) + (0.1X0.5)
(0.06X0.1) + (0.04X0.2) + (0.9X0.5)	(0.06X0.1) + (0.04X0.2) + (0.9X0.5)	•••	(0.06X0.1) + (0.04X0.2) + (0.9X0.5)

[attention output(3, 512)]

Transformer

- Multi-Head Attention
  - self attention을 병렬로 수행
  - 입력 문장의 다양한 측면을 병렬로 학습 → 속도 및 성능 향상
  - 최종 결과(새로운 단어 임베딩 벡터) → concat



weight 분할

	1	•••	8
W_q	64	•••	64
W_k	64	•••	64
W_v	64	•••	64

q, k, v 분할

	q_1	•••	q_8	k_1	•••	k_8	v_1	•••	v_8
I	64	•••	64	64	•••	64	64	•••	64
love	64	•••	64	64	•••	64	64	•••	64
you	64	•••	64	64	•••	64	64	•••	64

Transformer 실습

GPT(Generative Pre-Training)

- 2018년에 OpenAI가 공개한 언어 모델

1) 언어 모델

① 이전 단어들이 주어졌을 때 다음 단어로 어떤 단어가 적합한지 맞히는 과정에서 학습되는 모델

② 예시 "비행기를 타기 위해 □□□"

2) GPT1 : 1억1천7백만개의 가중치, GPT2 : 15억4천2백만 개의 가중치로 구성

3) 문장 생성에 강점을 가진 모델

4) Transformer의 디코더만 사용

BERT(Bidirectional Encoder Representations from Transformers)

- 2018년에 구글이 공개한 MASK 언어 모델

1) MASK 언어 모델

① 문장 내 특정 위치에 빈칸을 만들고 해당 빈칸에 적합한 단어가 무엇일지 분류하는 과정에서 학습되는 모델

② 예시: "비행기를 타기 위해 공항을 가는데 [MASK] 너무 막혀서 결국 비행기를 놓쳤다."

③ 사람의 경우, 수많은 단어와 문장을 듣고 쓰고 말하며 언어능력을 학습했기 때문에, 확률적으로 '차가'라는 단어가 가장 적합하다고 판단할 수 있음

④ 아이디어: 가중치가 매우 큰 언어 모델에 많은 양의 데이터를 학습

2) 1억1천만 ~ 3억4천5백만 개의 가중치로 구성, 총 33억 단어를 학습시킨 모델

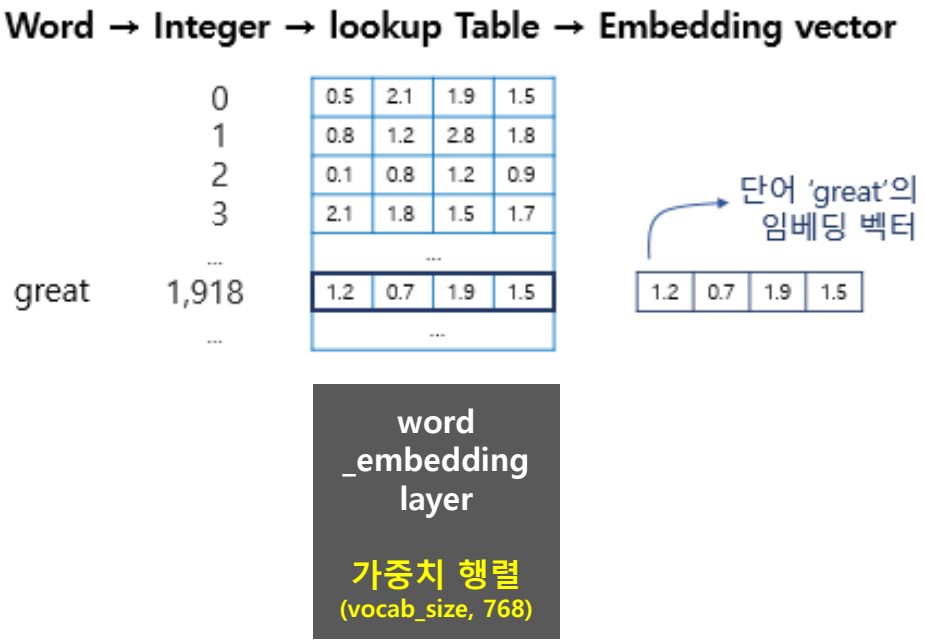
3) MASK 토큰 앞뒤 문맥을 모두 참고하는 양방향 언어 모델

4) 문장의 의미를 추출하는데 강점을 가진 모델

5) Transformer의 인코더만 사용

BERT(Bidirectional Encoder Representations from Transformers)

- 마스크 언어 모델(masked language model) 방식의 학습
  - 동작 과정(1) : text 입력 → 토큰화(BertTokenizer) + Masking
    → Token 임베딩 + Segment 임베딩 + Position 임베딩
    → 임베딩 행렬 생성
  - (예시) My dog is cute → ['My', 'dog', 'is', 'cute'] → [[CLS], 'My', [MASK], 'is', 'cute', [SEP]]

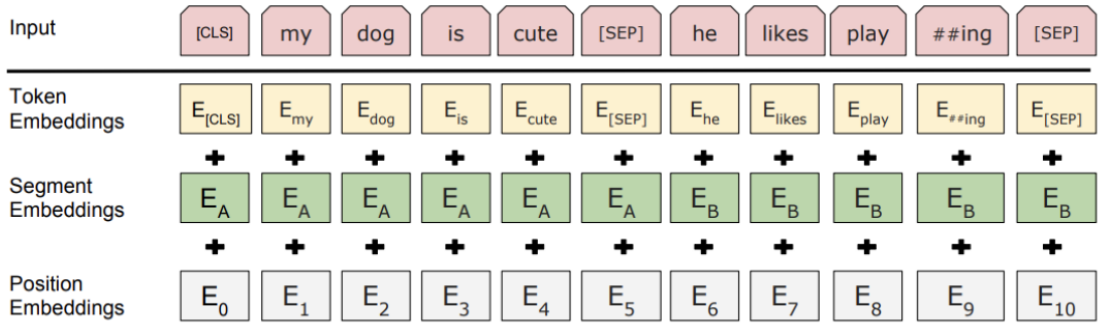


token	index
CLS	2
My	1000
MASK	4
is	1002
cute	1003
SEP	3

token	embedding vector
CLS	768
My	768
MASK	768
is	768
cute	768
SEP	768

BERT(Bidirectional Encoder Representations from Transformers)

- 마스크 언어 모델(masked language model) 방식의 학습
  - 동작 과정(1) : text 입력 → 토큰화(BertTokenizer) + Masking
    → Token 임베딩 + Segment 임베딩 + Position 임베딩
    → 임베딩 행렬 생성
  - (예시) My dog is cute → ['My', 'dog', 'is', 'cute'] → [[CLS], 'My', [MASK], 'is', 'cute', [SEP]]



token_type_embedding layer

가중치 행렬
(2, 768)

position_embedding layer

가중치 행렬
(max_length, 768)

BERT(Bidirectional Encoder Representations from Transformers)

- 마스크 언어 모델(masked language model) 방식의 학습
  - 동작 과정(2) : q, k, v 벡터 생성

token	embedding vector
CLS	768
My	768
MASK	768
is	768
cute	768
SEP	768

W_q	W_k	W_v
(768,768)	(768,768)	(768,768)
(768, 768 * 3)		

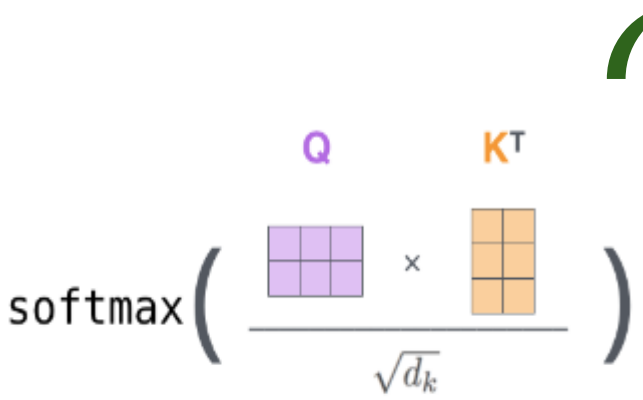


q	k	v
q(CLS)	k(CLS)	v(CLS)
q(My)	k(My)	v(My)
q(MASK)	k(MASK)	v(MASK)
q(is)	k(is)	v(is)
q(cute)	k(cute)	v(cute)
q(SEP)	k(SEP)	v(SEP)

BERT(Bidirectional Encoder Representations from Transformers)

- 마스크 언어 모델(masked language model) 방식의 학습
  - 동작 과정(3) : self-attention

→ 문장을 구성하고 있는 각 단어 간의 관계를 반영한 새로운 단어 임베딩 벡터 생성



token	embedding vector
CLS	768
My	768
MASK	768
is	768
cute	768
SEP	768

BERT(Bidirectional Encoder Representations from Transformers)

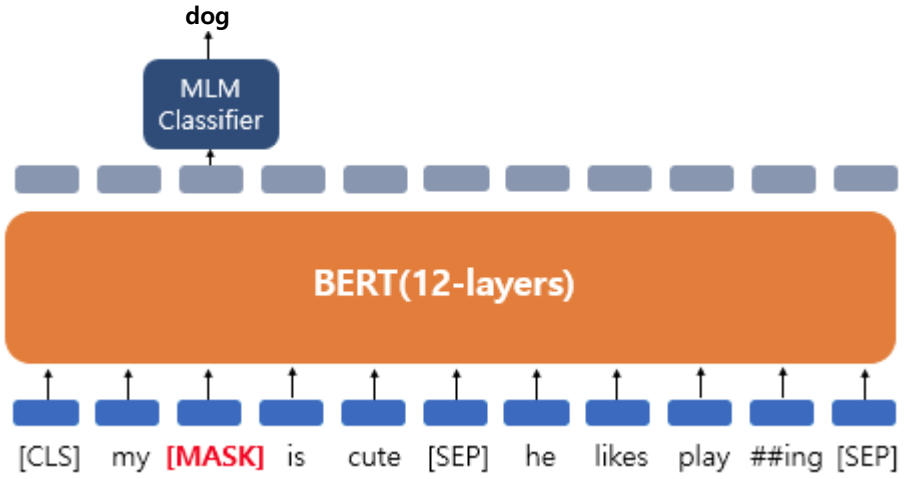
- 마스크 언어 모델(masked language model) 방식의 학습
  - 동작 과정(4) : 정답 토큰 예측 및 가중치 업데이트
  - 단어 분류를 위한 Dense layer 추가

1) 입력 : MASK 토큰에 대한 최종 임베딩 벡터

2) 출력 units = 학습 대상 전체 단어의 수

3) 출력 units의 결과 값 → softmax 함수 → 학습 대상 전체 단어에 대한 확률 계산

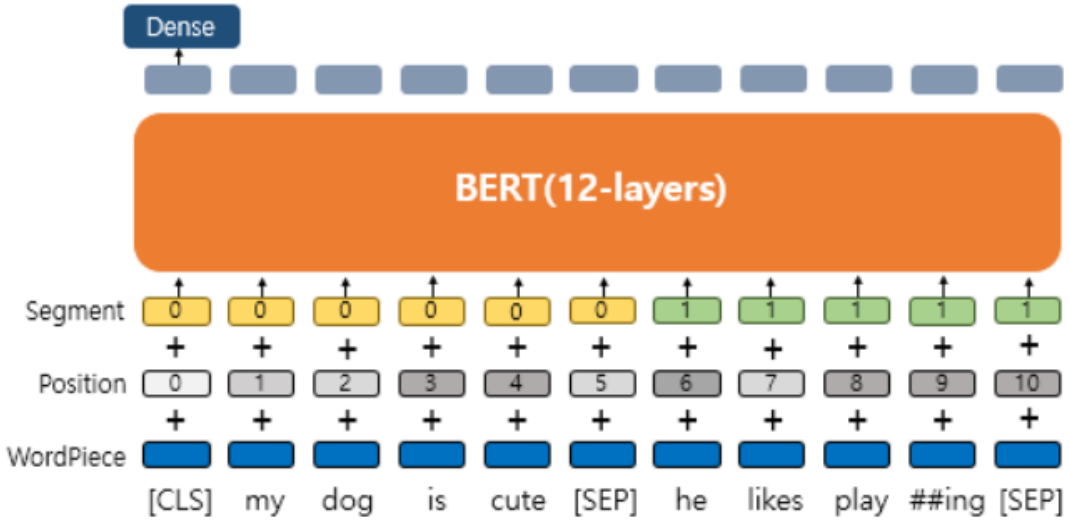
4) 정답 'dog'의 확률 증가, 나머지 단어의 확률 감소 → 학습 진행(가중치 업데이트)



BERT(Bidirectional Encoder Representations from Transformers)

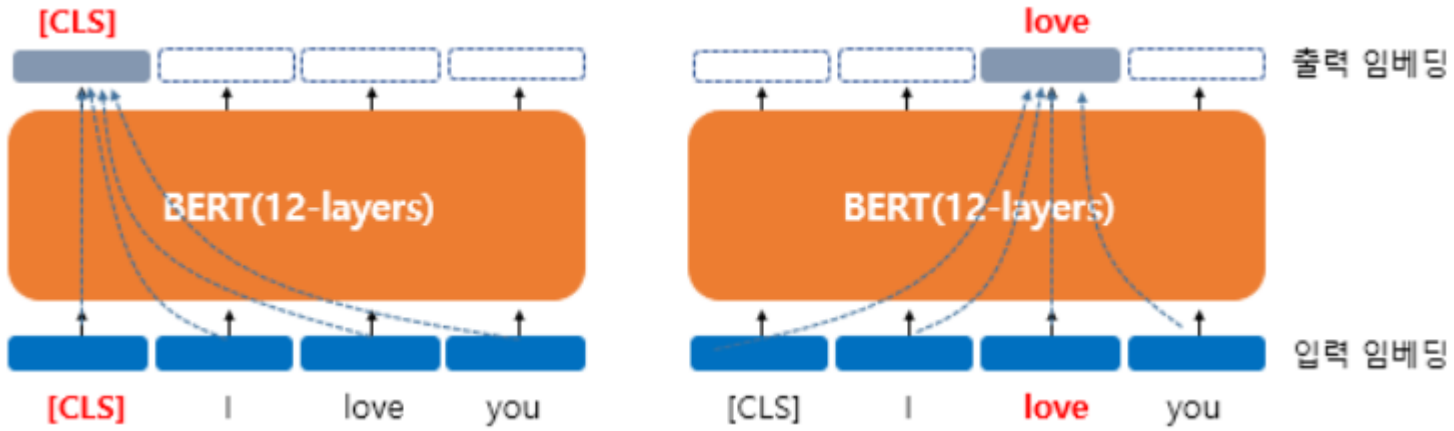
- 다음 문장 예측(Next Sentence Prediction, NSP) 방식의 학습
  - 두 개의 문장을 입력 후 두 문장이 이어지는 문장인지 아닌지를 맞추는 방식으로 학습
  - 두 개의 문장 입력 방식

    → 50:50 비율로 실제 이어지는 두 개의 문장과 랜덤으로 이어 붙인 두 개의 문장 입력
  - [CLS] token → 결과(Label) 출력 → Label : IsNextSentence or NotNextSentence



BERT(Bidirectional Encoder Representations from Transformers)

- BERT 출력 : contextual word embedding vector
  - 문장을 구성하는 각 단어들은 문장 내 모든 단어들을 참고한 문맥을 가진 임베딩 벡터가 됨



- 임베딩 벡터의 품질 우수 → 문장의 의미를 추출하는데 강점(읽고 이해하기)

사전 학습 / 전이 학습 / 추론

- 사전 학습(pre-training)
  - 많은 시간과 자원을 사용하여 방대한 양의 학습 데이터를 이용해서 모델의 파라미터(가중치)를 처음부터 학습시키는 과정

- 전이 학습(transfer learning)
  - 사전 학습된 모델을 사용자가 원하는 새로운 과제에 적용하는 과정
  - 미세 조정(fine-tuning)
    ① 사전 학습 모델이 가지고 있는 파라미터(가중치)를 사용자가 해결하고자 하는 과제에 대한 학습 데이터를 이용해서 추가적으로 학습하는 방법
    ② 사전 학습 모델이 가지고 있는 파라미터(가중치)를 초기값으로 사용해서 추가적인 학습을 통해 사용자가 가지고 있는 학습 데이터에 더 적합한 파라미터(가중치)를 얻는 것

사전 학습 / 전이 학습 / 추론

- 추론(Inference)
  - 학습(사전 학습 및 전이 학습)이 완료된 모델에 새로운 입력 데이터를 넣어 예측값(Label, Score 등)을 계산해내는 과정(학습 진행 X)



추론(Inference)

- HuggingFace 소개
  - 세계 최대의 오픈소스 AI 플랫폼
  - 학습이 완료된 다양한 딥러닝 모델들을 공유
  - Transformers 라이브러리 제공: 복잡한 딥러닝 모델을 단 몇 줄의 파이썬 코드로 불러와 바로 사용할 수 있게 해주는 업계 표준 오픈소스 도구 개발 및 배포
  - AI 배포 및 테스트 공간 (Spaces) 제공: 개발자가 만든 AI 모델을 코딩 없이 웹 브라우저에서 직접 테스트 해 볼 수 있는 체험 공간 제공
  - url: https://huggingface.co/

추론: HuggingFace 라이브러리 pipeline() 함수

- pipeline() 함수

  1) transformers 라이브러리의 가장 기본적인 함수

  2) 사전 학습된 모델을 연결하여 추론 작업을 편리하게 할 수 있도록 지원

- 사용법(예시)

  1) pipeline 함수 실행 : classifier = pipeline('text-classification' ,model)

  2) 문서 분류 실행 : preds = classifier(sentence)

텍스트 분류(Text Classification) 실습

- 실습 데이터 : 네이버 영화 리뷰 감성분석 데이터
  - 네이버 영화의 댓글을 모아 구성된 한국어 텍스트 → 컬럼 : id, document(리뷰), label(부정, 긍정)
  - 학습용 데이터 : 150,000개, 평가용 데이터 50,000개

- 실습 개요
  - HuggingFace 라이브러리 제공 사전 학습된 BERT 모델을 이용하여 리뷰의 감성을 예측하는 모델 생성
  - 학습용 데이터를 이용하여 전이 학습, 평가용 데이터를 이용하여 모델 평가 진행
  - 또는 사전 학습된 모델 이용 평가용 데이터를 이용하여 추론만 진행

텍스트 분류(Text Classification) 실습

- HuggingFace 라이브러리 제공 사전 학습된 모델 이용 전이 학습
  1) 학습용 / 검증용 / 평가용 데이터 생성
  2) Custom Dataset 클래스 함수 정의
  3) Custom Dataset 클래스 입력 데이터 생성
  4) 사전 학습된 BERT 모델 생성
  5) Trainer 환경 설정
  6) Trainer 생성
  7) 학습 실행
  8) 모델 평가

추론: HuggingFace 라이브러리 pipeline() 함수 실습

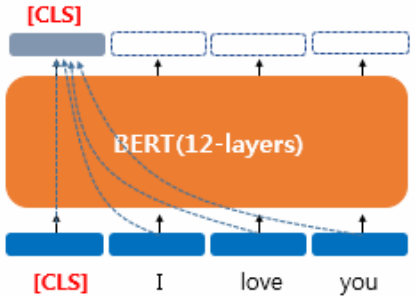
- 감성 분석(text-classification)
- 문장 생성(text generation)

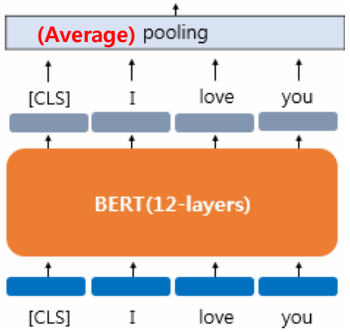
SentenceTransformer 모델

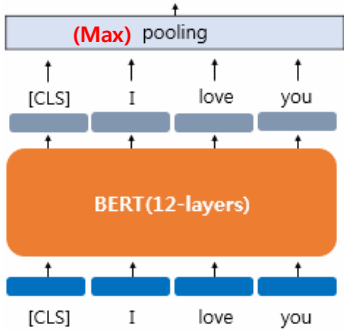
- SentenceTransformer : BERT를 기반으로 sentence embedding vector를 생성하는 모델
  - sentence embedding vector 생성 방법

    : BERT의 모든 출력 벡터에 대해서 평균값을 구하여 문장 벡터로 사용

    → 모든 단어의 의미 반영 → **"default"**



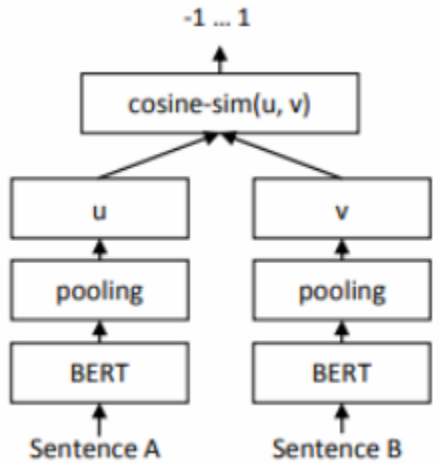




SentenceTransformer

- 용도

1) 두 문장 간의 유사도 계산



2) 문장 간의 유사도를 이용한 검색 엔진이나 ChatBot 개발 등에 활용

SentenceTransformer를 이용한 문장 유사도 측정 실습

- SentenceTransformer를 이용한 문장 유사도 측정
  1) 사전 학습된 한국어 SentenceTransformer 모델 생성
  2) 문장 임베딩 벡터 생성
  3) 코사인 유사도 측정

- 문장 유사도를 이용한 추천시스템

SentenceTransformer를 이용한 문장 유사도 측정 실습

문장 유사도(Text Similarity)를 이용한 ChatBot 실습

- 사용자 질문에 대한 응답 구현

  1) 사용자의 질문이 들어오면 해당 질문에 대한 문장 임베딩 벡터 생성

  2) 사용자 질문의 문장 임베딩 벡터 값과 챗봇 데이터의 임베딩 컬럼('embedding')에 저장해둔 예상 질문들의 문장 임베딩 벡터 값들을 이용, 코사인 유사도를 계산

  3) 코사인 유사도 값이 가장 높은 예상 질문 검색

  4) 해당 예상 질문과 짝이 되는 모범 답안 샘플 반환

문장 유사도(Text Similarity)를 이용한 ChatBot 실습


MISSION:
COMPLETE